

# L3: Convolutional Neural Network architectures for image analysis

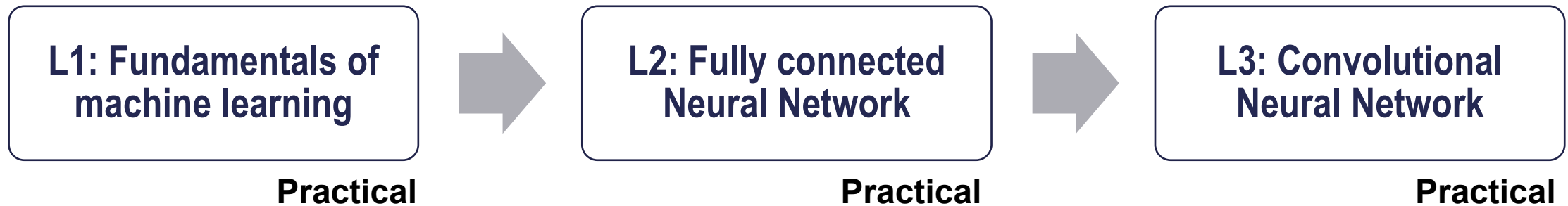
**EARTHLAB 2026**

Lecturer: Alice Cicirello ([ac685@cam.ac.uk](mailto:ac685@cam.ac.uk))

# Learning objectives of this lecture

## L3: Convolutional Neural Network architectures for image analysis

- Understand how to setup a **Convolution Neural Network (CNN)** through 5 steps: architecture setup, optimization setup, training, validation and testing
- Understand the **Residual Neural Network (ResNet)** architecture
- Understand the **U-Net** architecture



# Convolutional Neural Network (ConvNet/CNN) for image analysis

Handcrafted features, kernel embedding, metric learning

“Shallow” Machine Learning



Dataset composed only by images  
(e.g. stress distributions,  
spectrograms, NDT results,...)



$$\phi(x)$$

Mathematical  
representation of  
the image



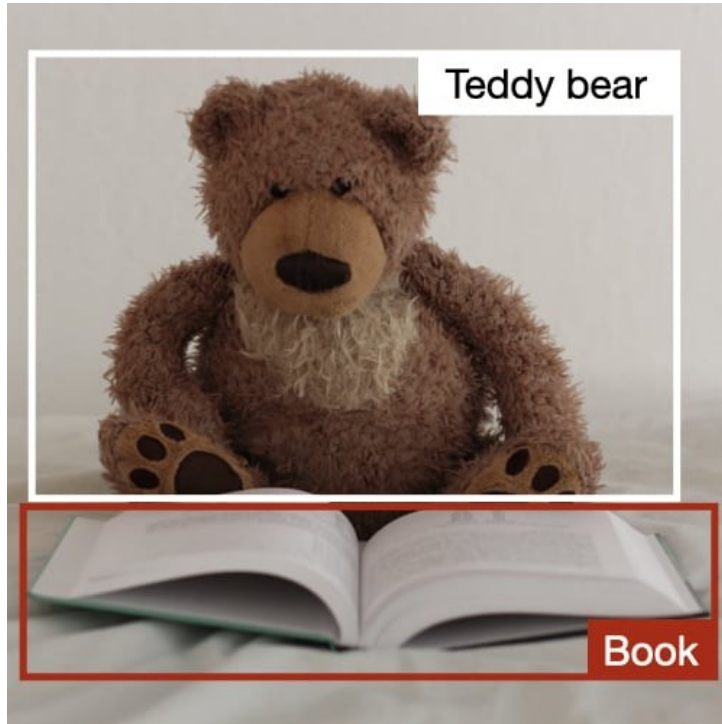
Build model predictor  
(tasks: classification,  
regression, **detection**,  
**segmentation**, ...)

CNN Deep Learning architectures (including ResNet, U-net)

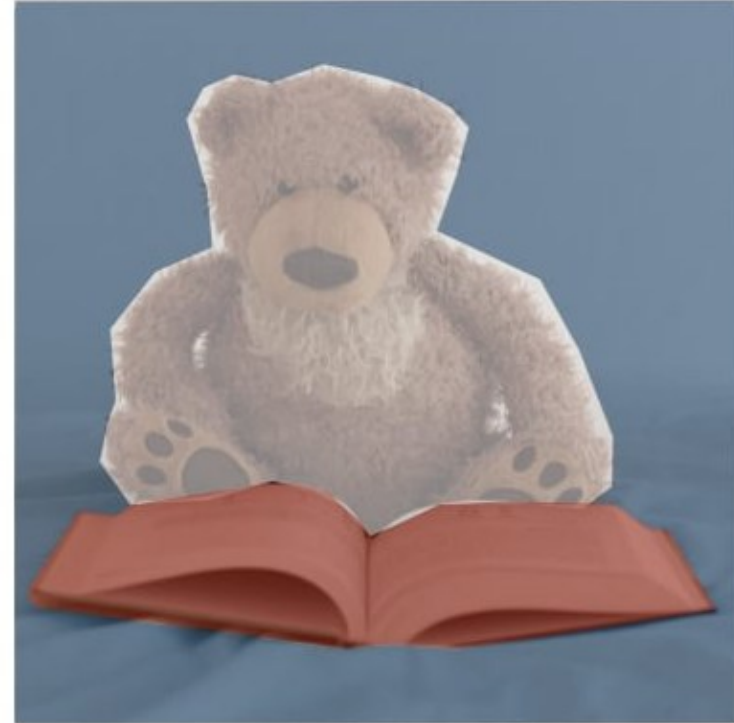
Deep Learning algorithms that from an input image, **use learnable weights and biases to automatically learn filters which highlight features** (e.g. edges)/objects in the image needed for model prediction.

**Initial developments 1950-60** → breakthrough: **AlexNet** by Krizhevsky et al. **2012** when for the first time a CNN was more successful than traditional, hand-crafted feature learning on ImageNet!

# Automatically understanding the content of a picture: **Detection** vs **Segmentation**



**Detection:** the computer detects and locates all the different objects within an image (output: labels and bounding boxes)



**Semantic segmentation** output: a high-resolution image, in which **each pixel** is classified.

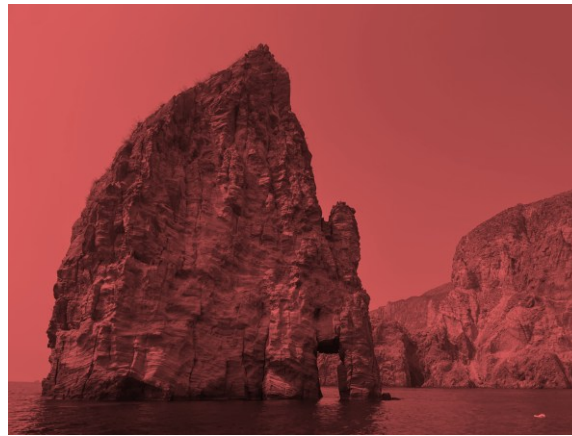
**Instance segmentation** output: classifying **each instance** of the same class separately (e.g. if there were two bears, each bear will be in the class bear, but classified separately using different colours)

Clearly, many useful applications in Engineering! **For example: autonomous vehicles!**

**Example pictures** from: <https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks>

# Images expressed as a stack of 2D Channels: the **3D tensor**

Height



Features: Red, Green, Blue (RGB) channels  $C=3$

Width



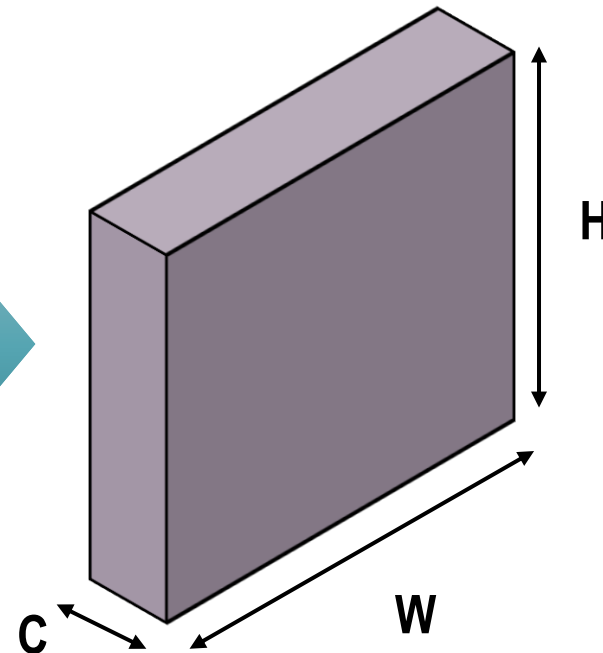
**Tensor: a multidimensional array.**

E.g. a vector in  $N$  dimensions  
= 1D array with  $N$  components

→ 2D tensor (a surface)

E.g. a matrix in  $N$  dimensions

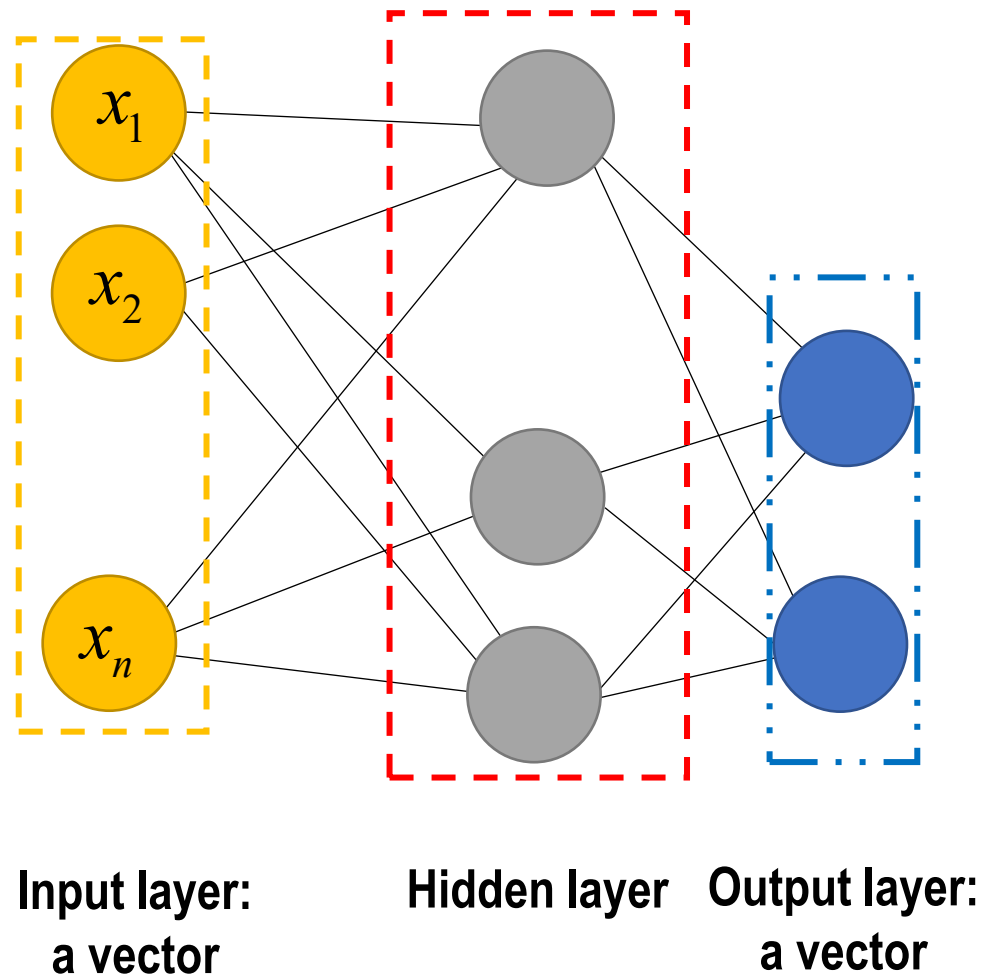
→ **3D tensor (a volume)**



Note; other channels can be used. **Even for  $C>3$ , this is still a 3D tensor representation!**

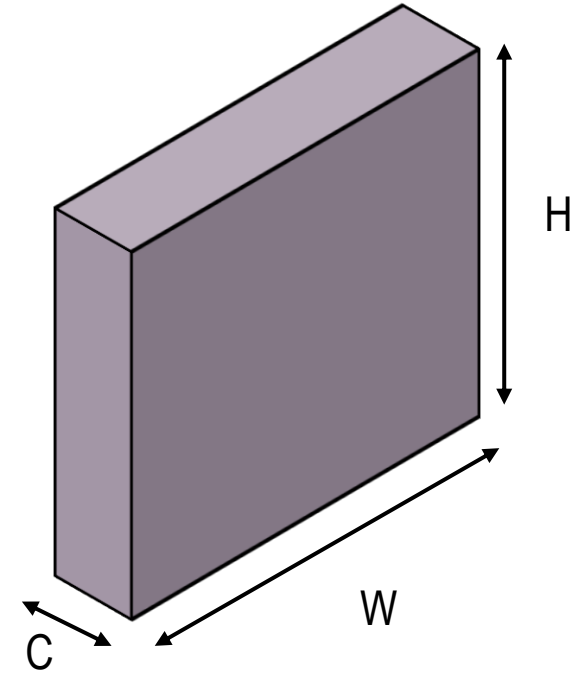


# Regular Fully Connected Neural Networks **do not scale well with images!**



CIFAR-10 dataset:

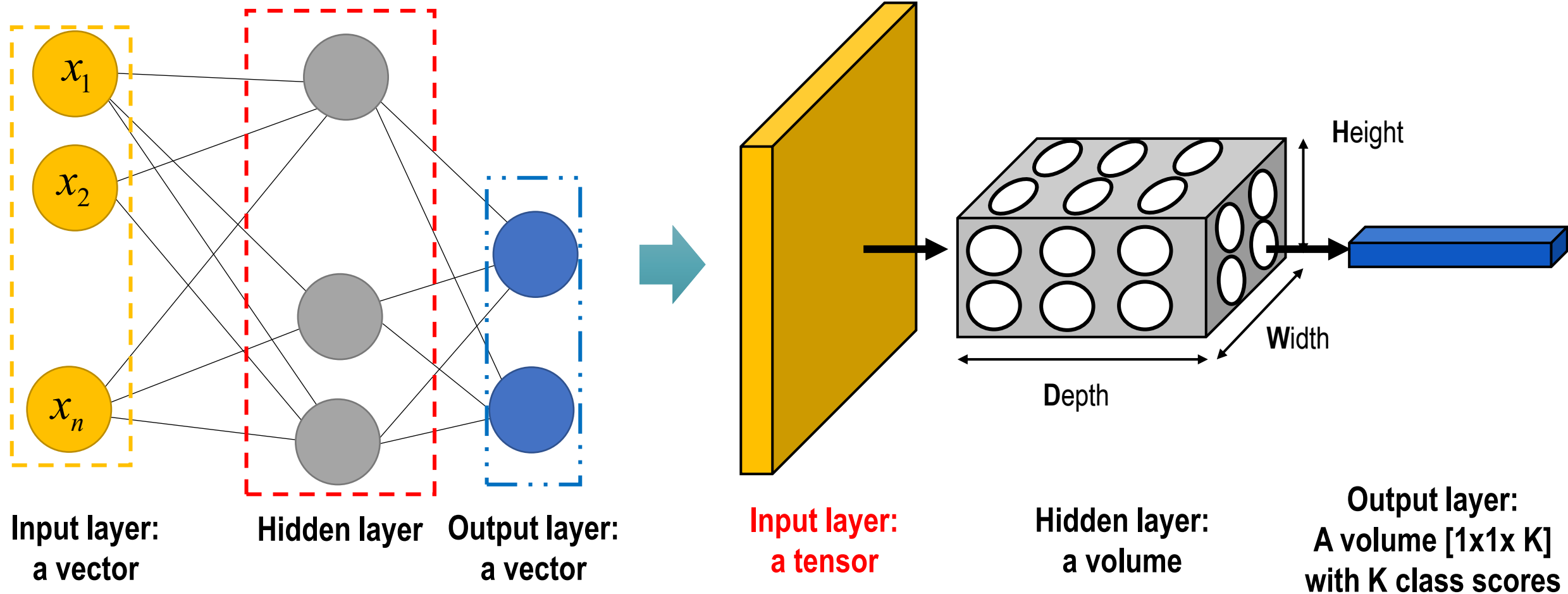
images of size 32 W x 32 H x 3 C



- We could flatten the image matrix of pixels to a long vector of pixel values.
- How **many weights** for a neuron in the first hidden layer of a FCNN? →  **$32 \times 32 \times 3 = 3072$  weights per node!!**
- Imagine with pictures of 256 W x 256 H!!!

**Moreover:** Forward pass would take a long time, lots of training data would be needed to reduce overfitting, small image variation could largely affect predictions, **spatial structure** in the image is lost!

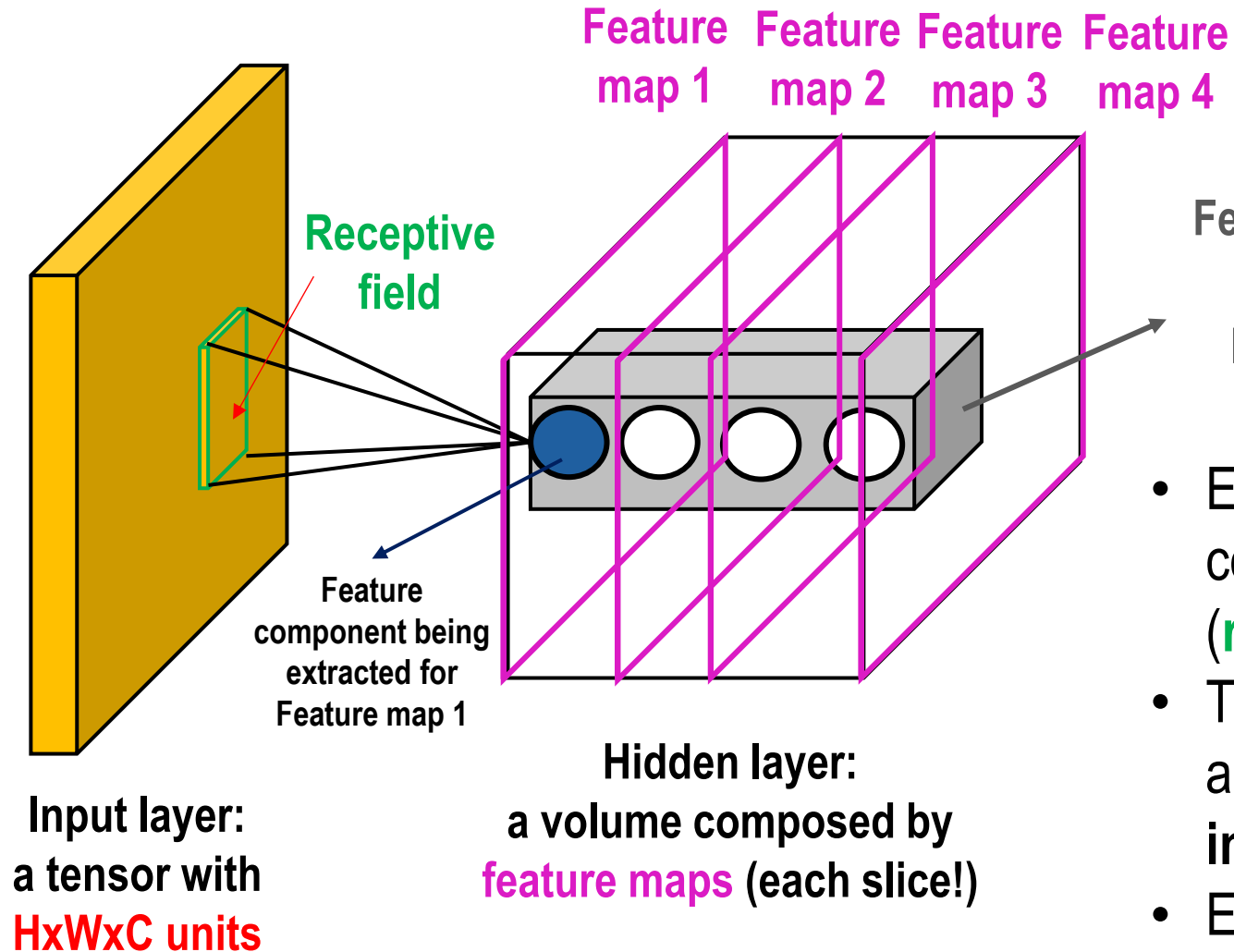
# Transforming 3D input volumes to 3D output volumes: **neurons arranged in 3D**



- Usually there are multiple layers between the input and output layer: **gradually increasing D, while decreasing H and W**
- CNNs preserve the spatial relationship between pixels by learning internal feature representations using small squares of input data.

# Not fully connected, but **local connectivity** – the **receptive field**

CNN uses **fewer parameters (weights and biases)** to learn compared to a fully connected network

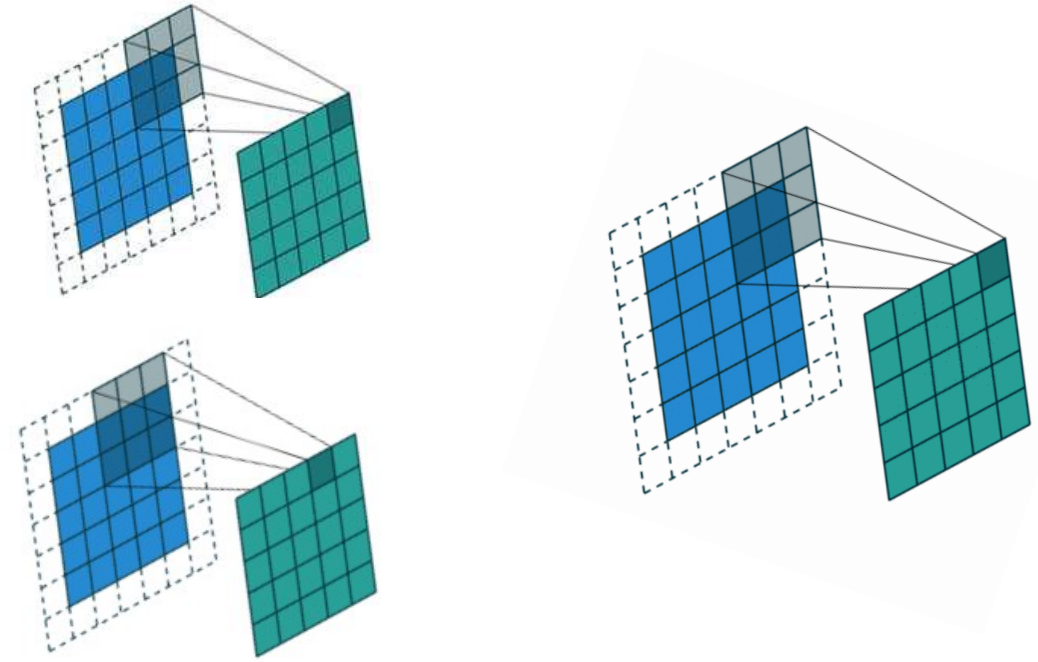
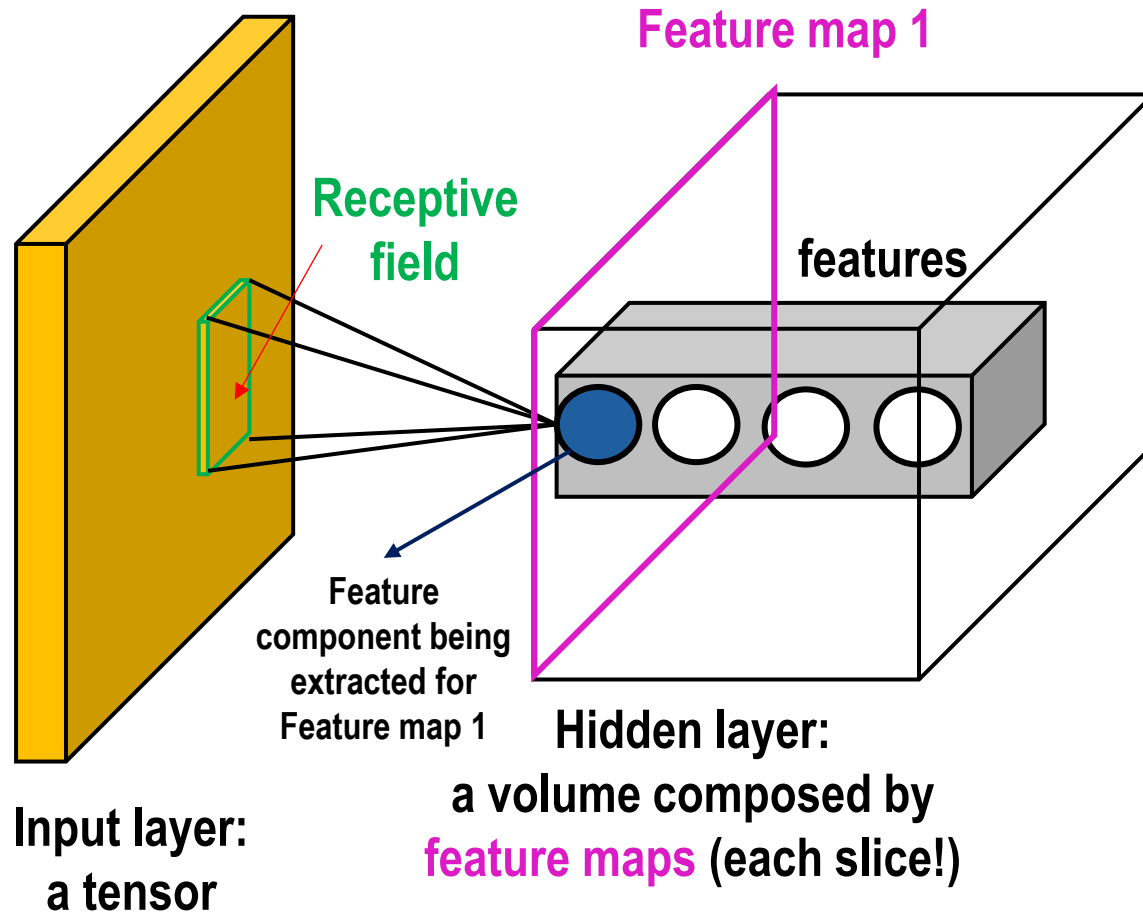


Feature components (hidden units)  
looking at the same region of the  
picture (same **receptive field**) for  
different **feature maps**

- Each **hidden unit (feature component)** connects to a small region of the input (**receptive field**)
- The **feature map** is the output of **one filter** applied to the previous layer (considering all **input channels**).
- Each element of the hidden layer is obtained via convolution



**Stride:** how many pixels the receptive field slides to cover the previous layer (considering all input channels)



- Stride=1, filter move one pixel at a time (example shown).
- Stride=2, (or uncommonly 3 or more) filters slides of 2 pixels at a time → will produce smaller output volumes spatially.

**Stride size** affects how the filter is applied to an image and the size of the resulting feature map

# Local connectivity and convolution in action: to obtain one feature map

Made up example: input image (5x5x3 units) and one output feature map (aka output channel), **stride=1**

Image expressed as R,G,B input channels (also known as input feature maps)

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 0   | 0   | 100 | 100 |
| 0   | 200 | 100 | 100 | 100 |
| 100 | 0   | 100 | 100 | 100 |
| 0   | 0   | 0   | 0   | 100 |
| 0   | 0   | 0   | 0   | 100 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 200 | 100 | 150 | 150 |
| 100 | 0   | 0   | 250 | 0   |
| 100 | 0   | 0   | 0   | 250 |
| 0   | 0   | 0   | 0   | 0   |
| 0   | 0   | 0   | 0   | 0   |

|     |     |   |   |     |
|-----|-----|---|---|-----|
| 200 | 100 | 0 | 0 | 0   |
| 100 | 200 | 0 | 0 | 0   |
| 100 | 100 | 0 | 0 | 0   |
| 0   | 0   | 0 | 0 | 100 |
| 0   | 0   | 0 | 0 | 100 |

**Convolution Kernels** → the actual filter! – **let's assign them for now!**

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

|   |   |    |
|---|---|----|
| 1 | 1 | -1 |
| 2 | 0 | 0  |
| 1 | 0 | 0  |

|   |    |    |
|---|----|----|
| 1 | 0  | 2  |
| 2 | 0  | 0  |
| 1 | -1 | -1 |

**A feature map** (3x3) obtained by summing the **convolved features units** of each input channel to be computed by considering input areas of 3x3 (same size as filter)

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

**+bias!**  
**Eg bias=1**

# Local connectivity – calculation for the first element of the output feature map

Receptive field (white area) in the input feature maps

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 0   | 0   | 100 | 100 |
| 0   | 200 | 100 | 100 | 100 |
| 100 | 0   | 100 | 100 | 100 |
| 0   | 0   | 0   | 0   | 100 |
| 0   | 0   | 0   | 0   | 100 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 200 | 100 | 150 | 150 |
| 100 | 0   | 0   | 250 | 0   |
| 100 | 0   | 0   | 0   | 250 |
| 0   | 0   | 0   | 0   | 0   |
| 0   | 0   | 0   | 0   | 0   |

|     |     |   |   |     |
|-----|-----|---|---|-----|
| 200 | 100 | 0 | 0 | 0   |
| 100 | 200 | 0 | 0 | 0   |
| 100 | 100 | 0 | 0 | 0   |
| 0   | 0   | 0 | 0 | 100 |
| 0   | 0   | 0 | 0 | 100 |

## Convolution Kernels

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

|   |   |    |
|---|---|----|
| 1 | 1 | -1 |
| 2 | 0 | 0  |
| 1 | 0 | 0  |

|   |    |    |
|---|----|----|
| 1 | 0  | 2  |
| 2 | 0  | 0  |
| 1 | -1 | -1 |

First element (unit) of each feature map (3x3)

\*  $100 \times 1 + 0 \times 0 + 0 \times (-1) + 0 \times 2 + 200 \times 0 + 100 \times (-2) + 100 \times 1 + 0 \times 0 + 100 \times (-1)$

|              |  |  |
|--------------|--|--|
| <b>-100*</b> |  |  |
|              |  |  |
|              |  |  |

+

|     |  |  |
|-----|--|--|
| 500 |  |  |
|     |  |  |
|     |  |  |

+

|     |  |  |
|-----|--|--|
| 400 |  |  |
|     |  |  |
|     |  |  |

+1

=

Output feature map

|            |  |  |
|------------|--|--|
| <b>801</b> |  |  |
|            |  |  |
|            |  |  |

# Local connectivity – calculation for **another element** of the output feature map

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 0   | 0   | 100 | 100 |
| 0   | 200 | 100 | 100 | 100 |
| 100 | 0   | 100 | 100 | 100 |
| 0   | 0   | 0   | 0   | 100 |
| 0   | 0   | 0   | 0   | 100 |

Receptive field in the input feature maps

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 200 | 100 | 150 | 150 |
| 100 | 0   | 0   | 250 | 0   |
| 100 | 0   | 0   | 0   | 250 |
| 0   | 0   | 0   | 0   | 0   |
| 0   | 0   | 0   | 0   | 0   |

|     |     |   |   |     |
|-----|-----|---|---|-----|
| 200 | 100 | 0 | 0 | 0   |
| 100 | 200 | 0 | 0 | 0   |
| 100 | 100 | 0 | 0 | 0   |
| 0   | 0   | 0 | 0 | 100 |
| 0   | 0   | 0 | 0 | 100 |

## Convolution Kernels

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

|   |   |    |
|---|---|----|
| 1 | 1 | -1 |
| 2 | 0 | 0  |
| 1 | 0 | 0  |

|   |    |    |
|---|----|----|
| 1 | 0  | 2  |
| 2 | 0  | 0  |
| 1 | -1 | -1 |

Another element (unit) of each feature map (3x3)

|              |      |  |
|--------------|------|--|
| <b>-100*</b> |      |  |
|              |      |  |
|              | -100 |  |

+

|     |   |  |
|-----|---|--|
| 500 |   |  |
|     |   |  |
|     | 0 |  |

+

|     |     |  |
|-----|-----|--|
| 400 |     |  |
|     |     |  |
|     | 100 |  |

+1

=

Output feature map

|            |   |  |
|------------|---|--|
| <b>801</b> |   |  |
|            |   |  |
|            | 1 |  |

In CNN we will **automatically learn** the **weights** (the entries of the convolution kernels) and the **bias** for each output feature map: these are the **trainable parameters**!

Receptive field in the input feature maps

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 0   | 0   | 100 | 100 |
| 0   | 200 | 100 | 100 | 100 |
| 100 | 0   | 100 | 100 | 100 |
| 0   | 0   | 0   | 0   | 100 |
| 0   | 0   | 0   | 0   | 100 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 200 | 100 | 150 | 150 |
| 100 | 0   | 0   | 250 | 0   |
| 100 | 0   | 0   | 0   | 250 |
| 0   | 0   | 0   | 0   | 0   |
| 0   | 0   | 0   | 0   | 0   |

|     |     |   |   |     |
|-----|-----|---|---|-----|
| 200 | 100 | 0 | 0 | 0   |
| 100 | 200 | 0 | 0 | 0   |
| 100 | 100 | 0 | 0 | 0   |
| 0   | 0   | 0 | 0 | 100 |
| 0   | 0   | 0 | 0 | 100 |

Convolution Kernels = **weights to be learned** – note: the same weights are used across **each region** (**weight sharing**)

|        |        |        |
|--------|--------|--------|
| W1,1,1 | W1,2,1 | W1,3,1 |
| W2,1,1 | W2,2,1 | W2,3,1 |
| W3,1,1 | W3,2,1 | W3,3,1 |

|        |        |        |
|--------|--------|--------|
| W1,1,2 | W1,2,2 | W1,3,2 |
| W2,1,2 | W2,2,2 | W2,3,2 |
| W3,1,2 | W3,2,2 | W3,3,2 |

|        |        |        |
|--------|--------|--------|
| W1,1,3 | W1,2,3 | W1,3,3 |
| W2,1,3 | W2,2,3 | W2,3,3 |
| W3,1,3 | W3,2,3 | W3,3,3 |

Another element (unit) of each feature map (3x3)

|        |  |  |
|--------|--|--|
| M1,1,1 |  |  |
|        |  |  |
|        |  |  |

+

|        |  |  |
|--------|--|--|
| M1,1,2 |  |  |
|        |  |  |
|        |  |  |

+

|        |  |  |
|--------|--|--|
| M1,1,3 |  |  |
|        |  |  |
|        |  |  |

+ Bias

=

Output feature map

|       |  |  |
|-------|--|--|
| OM1,1 |  |  |
|       |  |  |
|       |  |  |

# Example to consolidate concepts

- **Input features maps:**  $32 \times 32 \times 5 \rightarrow W \times H \times \text{Depth}$
- **Convolution kernel size to extract each feature map:**  $3 \times 3 \times 5$
- Obtain: **8 feature maps**

How many input channels?

**5**

How many output channels?

**8 output feature maps**

How many **units** in the input layer?

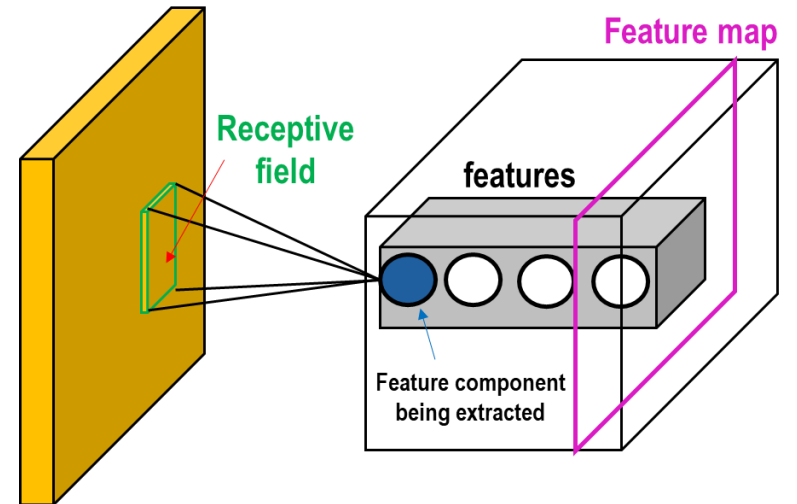
$32 \times 32 \times 5$

How many **total trainable parameters** are there?

$(3 \times 3 \times 5 + 1) \times 8$

How many **hidden units** looking at the same region of a picture?

**8!**





# Advantages of CNN

Handcrafted features, kernel embedding, metric learning

“Shallow” ML: eg FCNN



Dataset composed only by images  
(e.g. stress distributions,  
spectrograms, NDT results,...)



$$\phi(x)$$

Tensor  
representation of  
the image

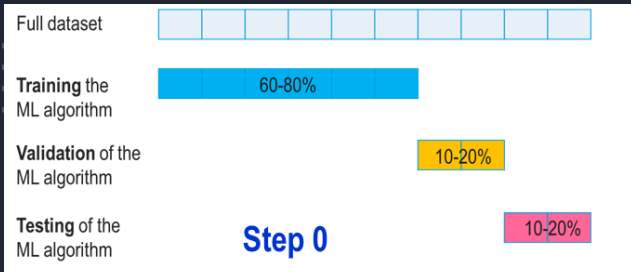


Build model predictor  
(tasks: classification,  
regression, detection,  
segmentation, ...)

**CNN Deep Learning architectures** (including ResNet, U-net)

- Automatically learn and generalize features from the input domain
- Use less learnable weights and biases than FCNN
- Designed to be invariant to object position and distortion of the scene

To learn the weights, biases and hyperparameters: same steps discussed in the **FCNN**



**Main differences!**



**Step 1: CNN architecture setup**



**Step 2: Optimization setup: AutoDiff to obtain the gradients**



**Step 3: Network training**



**Step 4: Tuning of Network parameters on validation data set**



**Step 5: Test on the CNN**

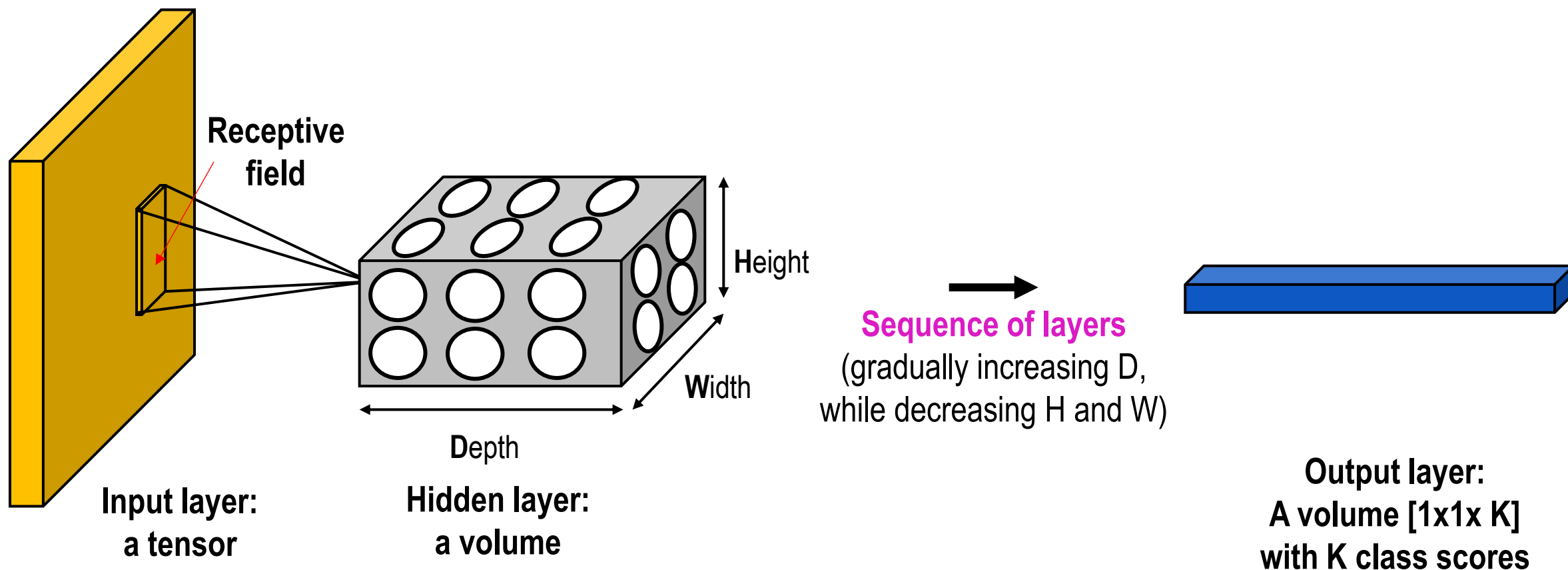
Steps to setup a  
Convolution  
Neural Network  
(CNN)

Look again at the  
material for FCNN



## Step 1: CNN architecture setup

# Hyperparameters & sequence of layers

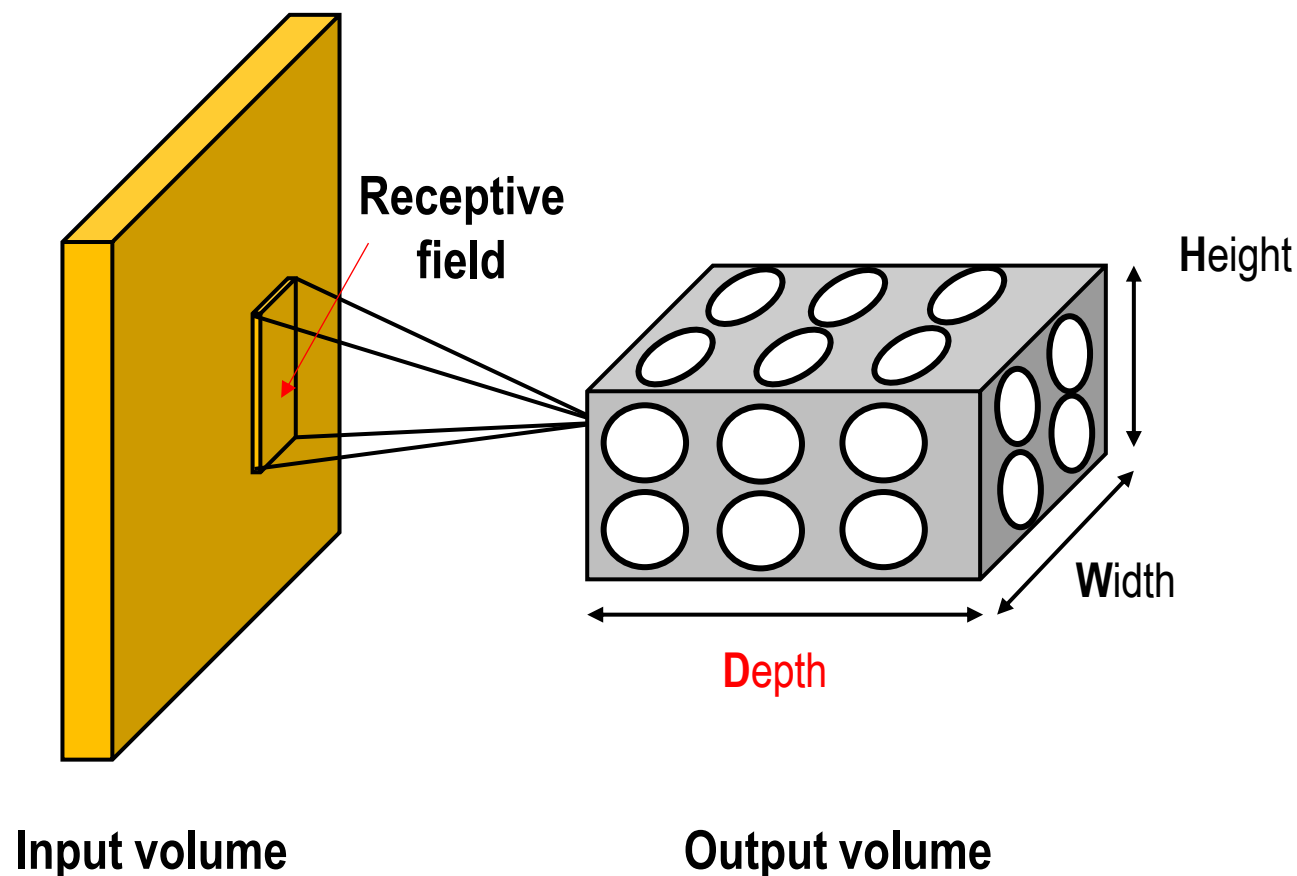


- **Hyperparameters** define the spatial arrangement **from the input volume to the output volume: Depth, Stride** (already discussed) and **Zero-padding**
- The possible **sequence of layers** between the input and the output layers will affect the performance of the CNN on a task: we will first look at each layer in isolation



## Step 1: CNN architecture setup

# Defining the spatial arrangement: **depth hyperparameter**



**Depth: number of filters** we would like to use in the output volume (each filter will learn something different from the input).

**Depth column:** a set of neurons all looking at the same region of the input (aka ***fibre***) – the coordinates are defined in terms of W and H

### Example:

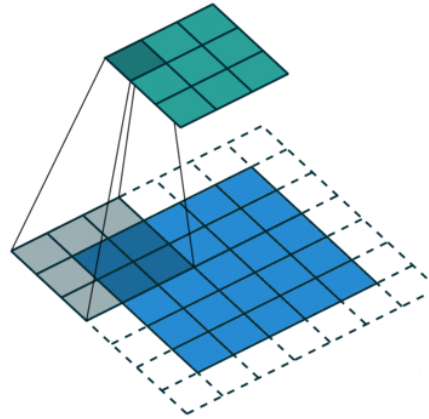
- First Convolutional Layer with raw image as input
- Each neuron along the depth dimension will activate in presence of various features (e.g. edges, or blobs of color).



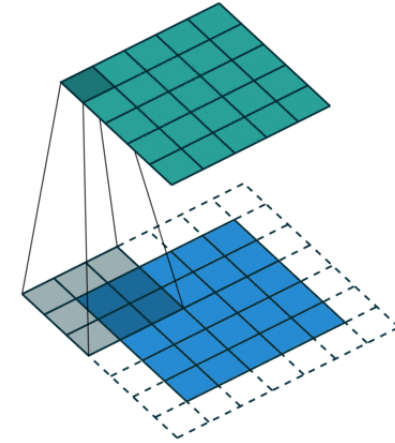
## Step 1: CNN architecture setup

## Defining the spatial arrangement: **zero-padding hyperparameter**

|   |     |     |     |     |     |   |
|---|-----|-----|-----|-----|-----|---|
| 0 | 0   | 0   | 0   | 0   | 0   | 0 |
| 0 | 100 | 0   | 0   | 100 | 100 | 0 |
| 0 | 0   | 200 | 100 | 100 | 100 | 0 |
| 0 | 100 | 0   | 100 | 100 | 100 | 0 |
| 0 | 0   | 0   | 0   | 0   | 100 | 0 |
| 0 | 0   | 0   | 0   | 0   | 100 | 0 |
| 0 | 0   | 0   | 0   | 0   | 0   | 0 |



Stride 2



Stride 1

- Pad the input volume with zeros around the border:
  - To artificially increase the size of the input image, without adding information.
  - To preserve the spatial size of the input volume so the **input and output width and height are the same: needed to design deeper networks.**
  - To ensure each pixel in the image is given an opportunity to be at the centre of the filter → important for pixels at the edge!



## Step 1: CNN architecture setup

## Core Layers of a CNN



**Convolutional layer**



**RELU layer**



**Average and Max pooling layer**



**Downsampling (Global Pooling Layer)**



**Fully connected layer**

$$\frac{x - \mu}{\sigma}$$

— Mean  
— Standard Deviation

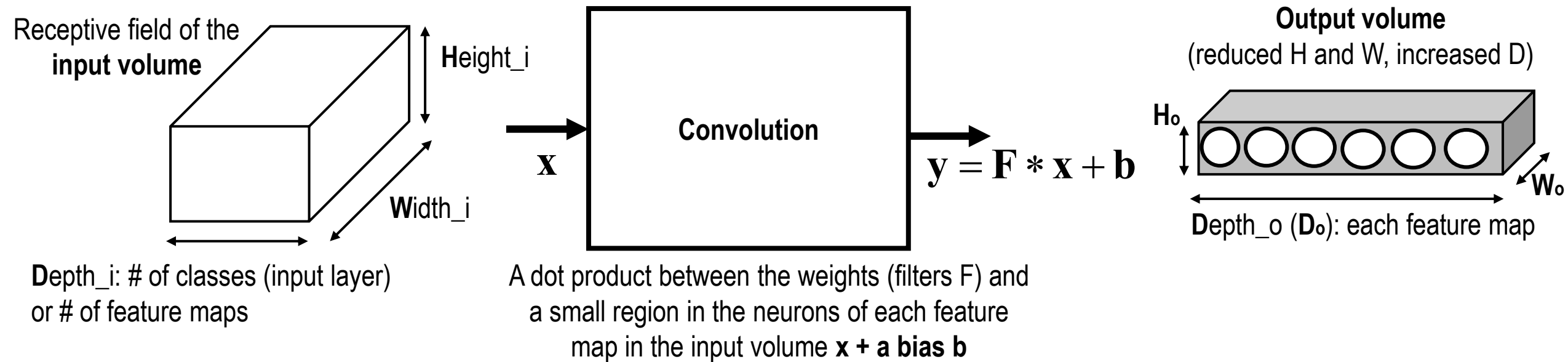
**Normalization layer & Batch Normalization**



## Convolutional layer (CONV)

**Compute the output of neurons** that are connected to local regions in the input

- The CONV layer produces a set of **linear activations** – convolution is a linear operator!
- **The CONV layer is learned (weights and biases)**
- Each convolution layer consists of several convolutions, each with its own kernel yielding a feature map



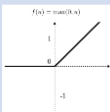
Hyperparameters to be specified:

- Spatial extent (HxW) of the receptive field (usually a square patch).
- Depth (number of filters)
- Stride
- Zero-padding

### Note:

- weights obtained from the first CONV layer are the most interpretable
- When visualizing the weights deeper in the network, if noisy patterns and sharp variations appear:
  - The network might need to be trained longer
  - A low regularization could have led to overfitting

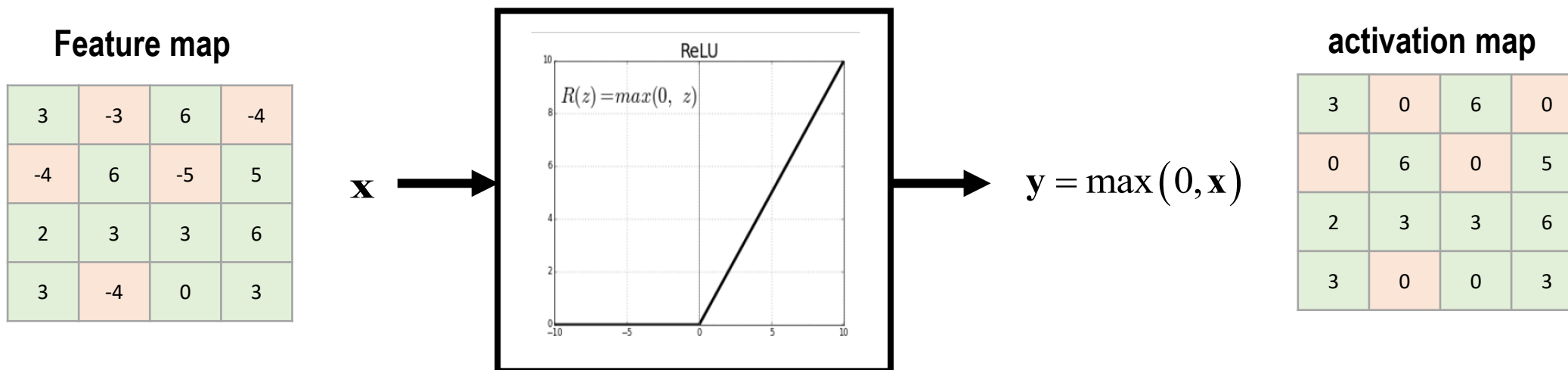




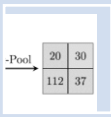
## ReLU layer (ReLU)

Introducing **nonlinearities** using activation functions:  
typically, **rectified linear (ReLU)**

Feature maps are passed through a nonlinear activation function (this is **specified without the need of hyperparameters!!**) → also known as **detector stage**.



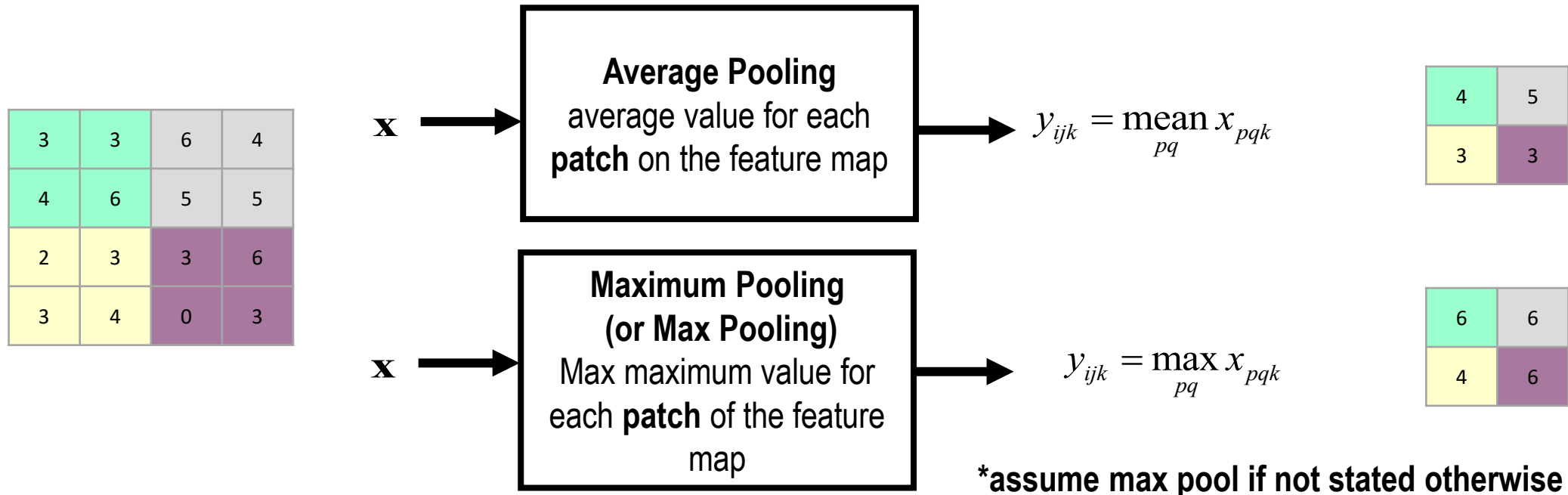
- This layer does not change the size of the volume, but applies elementwise a nonlinearity
- Usually, it is paired with a Convolutional Layer (**CONV-RELU stack**)
- **Note:** activation maps can be visualized to **investigate high learning rates:** occurring when some activation maps might be all zero for many different inputs (i.e. dead filters)



## Average and Max pooling layer (POOL)

**Down-sample patches** of the previous layer's feature map along the spatial dimensions (W, H)

- The pooling operation is **specified (requires hyperparameters)** → important to control overfitting
  - Usually, it is located in-between successive CONV-RELU layers
  - It operates independently on every depth slice of the input and resizes it spatially (i.e. depth dimension unchanged)
- Most common:** size (p,q) of 2x2, stride=2, downsampling every depth slice in the input by 2 along both width and height.



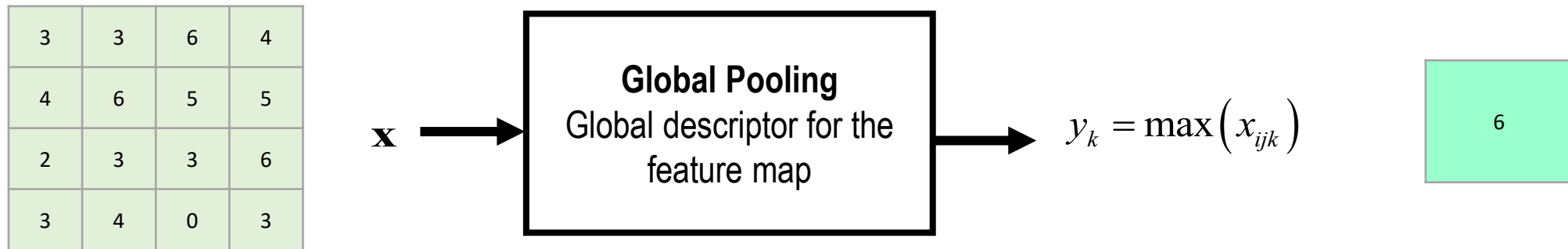
“In a nutshell, the reason is that features tend to encode the spatial presence of some pattern or concept over the different tiles of the feature map (hence, the term feature map), and it’s more informative to look at the maximal presence of different features than at their average presence”. Page 129, Deep Learning with Python, 2017.



## Downsampling (Global Pooling Layer)

Down-sampling the entire feature map into a single value

- This pooling operation is **specified (but no hyperparameters)**, rather than learned.



- Used to aggressively summarize the presence of a feature in an image.
- Sometimes used as an alternative to using a fully connected layer to transition from feature maps to an **output prediction for the model**.



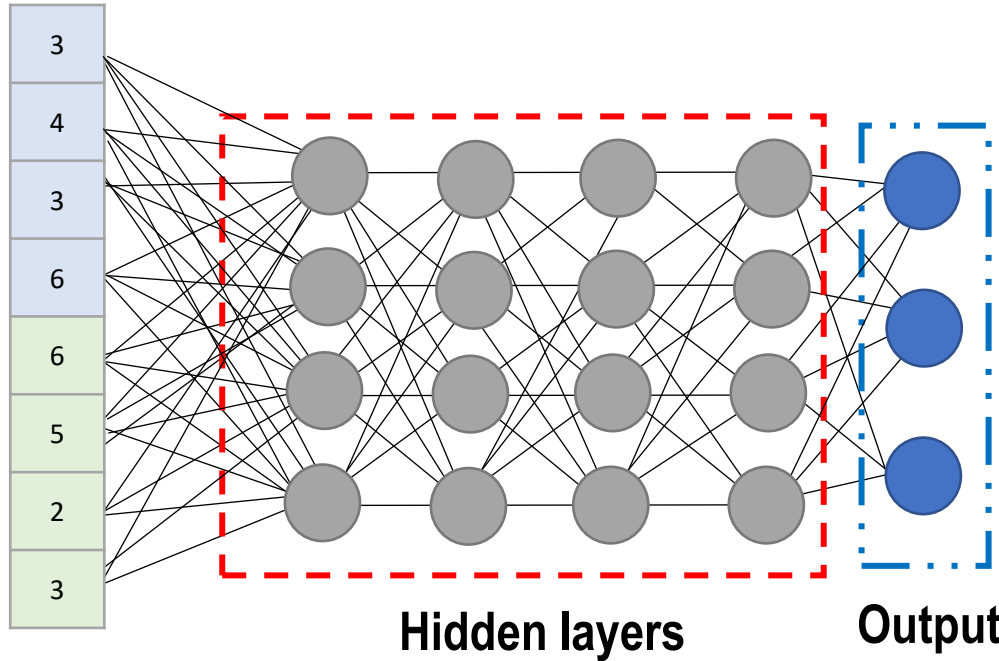
# Fully connected layer (FC or Dense)

## From feature maps to output predictions

square feature maps

|   |   |
|---|---|
| 3 | 3 |
| 4 | 6 |

|   |   |
|---|---|
| 6 | 2 |
| 5 | 3 |



**Reshape**

(sometimes referred to as a **flatten layer** before FC)

**Fully connected layer (FC or Dense)**

- This layer is usually referred to as **FC** or **Dense**

- High level features used as input to FC can be originated from the output CONV layer or of another FC layer.
- **A stack of FC layers** is usually used at the **end** of a CNN architecture to fully exploit FCNN architectures for transitioning from feature maps to an output prediction (e.g. final class score)
- The results quality depends on:
  - Quality of the information in the fully connected layer
  - **Architecture and hyperparameters selection to learn weights and biases → this layer is learned.**

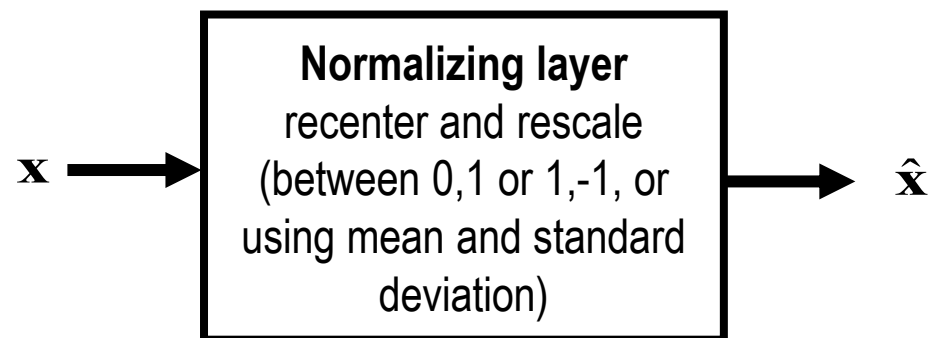
$$\frac{x - \mu}{\sigma}$$

Mean  
Standard Deviation

## Normalization layer & Batch Normalization

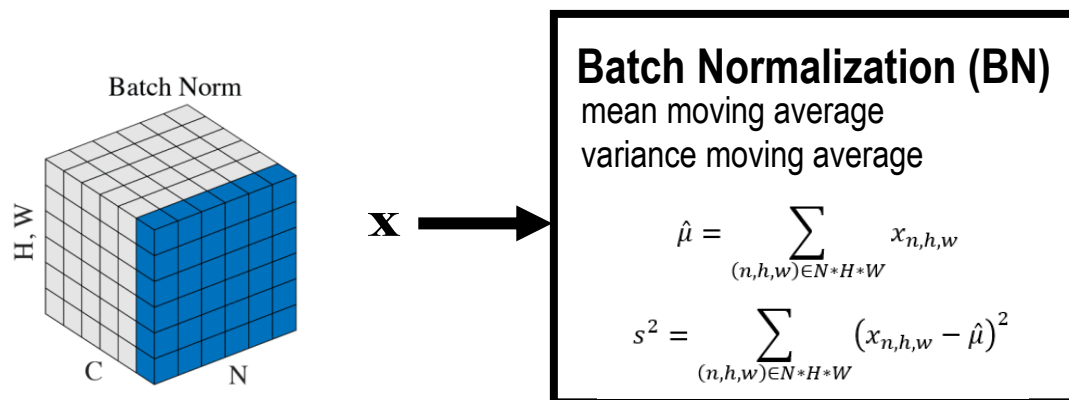
To create a set of features that are on the **same scale as each other** → improving performance

- Not only for inputs -Normalization can be carried out at intermediate levels.. With care!
- This layer is usually introduced between FC layers, and sometimes between CONV and RELU layers



“For convolutional layers, we additionally want the normalization to obey the convolutional property – so that **different elements of the same feature map, at different locations, are normalized in the same way**” -

<https://arxiv.org/abs/1502.03167>, Ioffe & Szegedy



$$x_{n,h,w}^{normalized} = \frac{x_{n,h,w} - \hat{\mu}}{s}$$

**For each BN, two hyperparameters are learned using backprop during training**

Statistics over all locations of the same channel



- Since Batch normalization has a regularizing effect, do not use it together with dropout
- Avoid too small batches → use normalization layer

# Steps to setup a Convolution Neural Network (CNN)

Look again at the material for FCNN



**Step 1: CNN architecture setup**



**Step 2: Optimization setup: AutoDiff to obtain the gradients**



**Step 3: Network training**



**Step 4: Tuning of Network parameters on validation data set**



**Step 5: Test on the CNN**



## Step 2: Optimization setup: AutoDiff to obtain the gradients

Backpropagating the errors from the last to the first layer of the network

AutoDiff returns the exact value (**up to machine accuracy**) for a gradient: been around since **1960!**

### PSEUDO code CNN - forward

#### Require:

CNN architecture specification

an image (tensors  $\mathbf{X}_0$  classes C),

Weights initialization:  $\mathbf{W}_1, \dots, \mathbf{W}_N$

**for**  $j=1, \dots, n$  CONV (for each CONV layer)

**do**  $\mathbf{x}_j = f_j(\mathbf{x}_{j-1}, \mathbf{w}_j)$

**End for**

**Evaluate the loss** to assess the quality of the CNN prediction

- Once the forward pass is completed, the backward pass requires to evaluate the **gradients** of the loss (a scalar function!) wrt CNN parameters!
- These derivative can be automatically computed with **AutoDiff!**
- Backprop is a special case of AutoDiff

$$\frac{d(f_n \dots f_{i+1})}{d\mathbf{x}_i} = \frac{df_n}{d\mathbf{x}_{n-1}} \frac{df_{n-1}}{d\mathbf{x}_{n-2}} \dots \frac{df_1}{d\mathbf{x}_0}$$



## Step 2: Optimization setup: AutoDiff to obtain the gradients

## The key elements of AutoDiff



- Numerical differentiation (central difference)
  - Symbolic differentiation → we do not have an algebraic expression of the gradient
- 
- AutoDiff keeps track of calculations performed in a program using a **computational graph**
  - The function is **decomposed into a sequence of elementary arithmetic operation and functions**
  - Chain rule is used to work out the function's derivatives: **all variables are tensors in 1D (vectors)**





## Step 2: Optimization setup: AutoDiff to obtain the gradients

## Automatic differentiation in action: sequence of steps & computational graph

(adapted from Neto, ECEN 689: Scientific Machine Learning,  
Lecture 4: Automatic Differentiation, 2023)

$$f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$$

The function can be computed as a  
sequence of elementary steps:

$$s_{-1} = x_1$$

$$s_0 = x_2$$

$$s_1 = \ln(s_{-1})$$

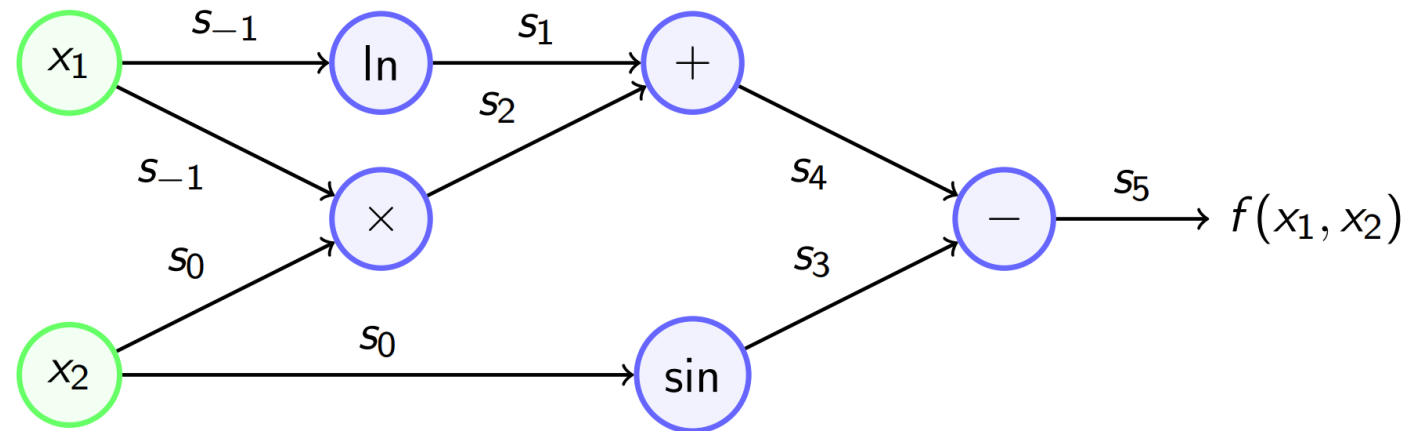
$$s_2 = s_{-1} s_0$$

$$s_3 = \sin(s_0)$$

$$s_4 = s_1 + s_2$$

$$s_5 = s_4 - s_3$$

### Corresponding computational graph



← Operations to compute values of the function



## Step 2: Optimization setup: AutoDiff to obtain the gradients

## Autodiff: forward vs backward passes

(adapted from Neto, ECEN 689: Scientific Machine Learning,  
Lecture 4: Automatic Differentiation, 2023)

Use AutoDiff to compute the gradient of  $y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$

**Forward pass:** compute derivative in single pass

$$s_{-1} = x_1$$

$$s_0 = x_2$$

$$s_1 = \ln(s_{-1})$$

$$s_2 = s_{-1} s_0$$

$$s_3 = \sin(s_0)$$

$$s_4 = s_1 + s_2$$

$$s_5 = s_4 - s_3$$

$$\dot{s}_{-1} = \dot{x}_1$$

$$\dot{s}_0 = \dot{x}_2$$

$$\dot{s}_1 = \dot{s}_{-1} / s_{-1}$$

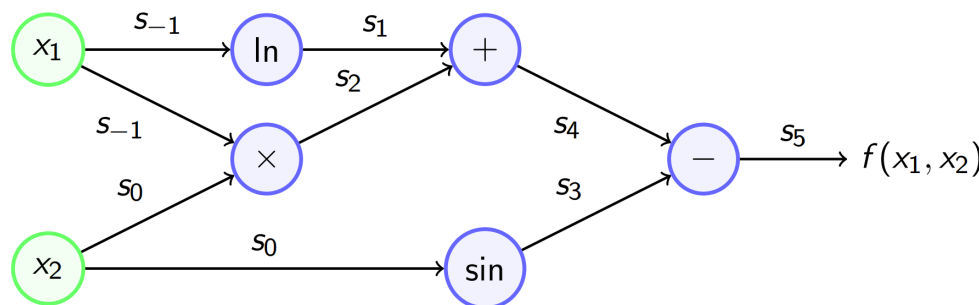
$$\dot{s}_2 = \dot{s}_{-1} s_0 + s_{-1} \dot{s}_0$$

$$\dot{s}_3 = \cos(s_0) \dot{s}_0$$

$$\dot{s}_4 = \dot{s}_1 + \dot{s}_2$$

$$\dot{s}_5 = \dot{s}_4 - \dot{s}_3$$

**Backward pass:** compute derivative in single pass (**needs values from forward pass!**)



$$\bar{s}_i = \sum_{j \in \text{successor of } i} \bar{s}_j \frac{\partial \bar{s}_j}{\partial s_i}$$

← Operations to compute derivatives →

$$\bar{s}_5 = \frac{\partial y}{\partial s_5}$$

$$\bar{s}_4 = \bar{s}_5 \frac{\partial \bar{s}_5}{\partial s_4}$$

$$\bar{s}_3 = \bar{s}_5 \frac{\partial \bar{s}_4}{\partial s_3}$$

$$\bar{s}_2 = \bar{s}_4 \frac{\partial \bar{s}_4}{\partial s_2}$$

$$\bar{s}_1 = \bar{s}_4 \frac{\partial \bar{s}_4}{\partial s_1}$$

$$\bar{s}_0 = \bar{s}_2 \frac{\partial \bar{s}_2}{\partial s_0} + \bar{s}_3 \frac{\partial \bar{s}_3}{\partial s_0}$$

$$\bar{s}_{-1} = \bar{s}_1 \frac{\partial \bar{s}_1}{\partial s_{-1}} + \bar{s}_2 \frac{\partial \bar{s}_2}{\partial s_{-1}}$$

# 3 CNN Architectures that introduced important changes: AlexNet, ResNET and U-Net

Image to vector

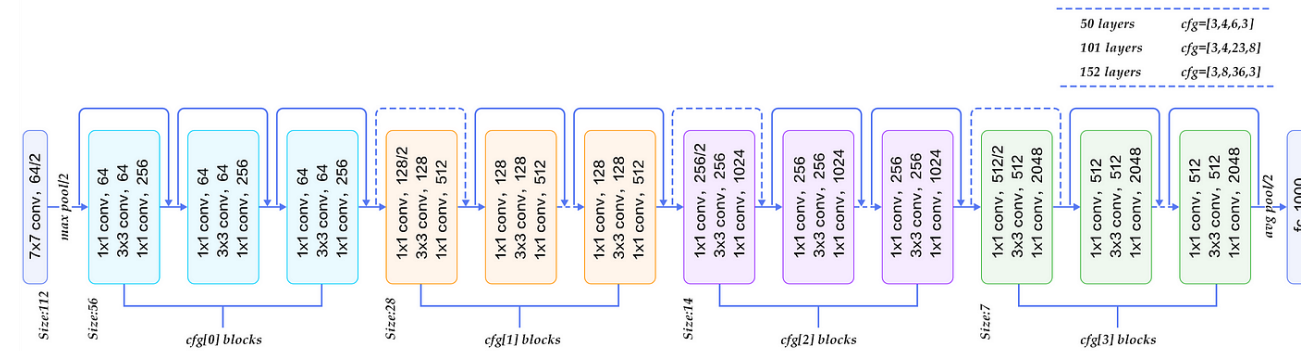
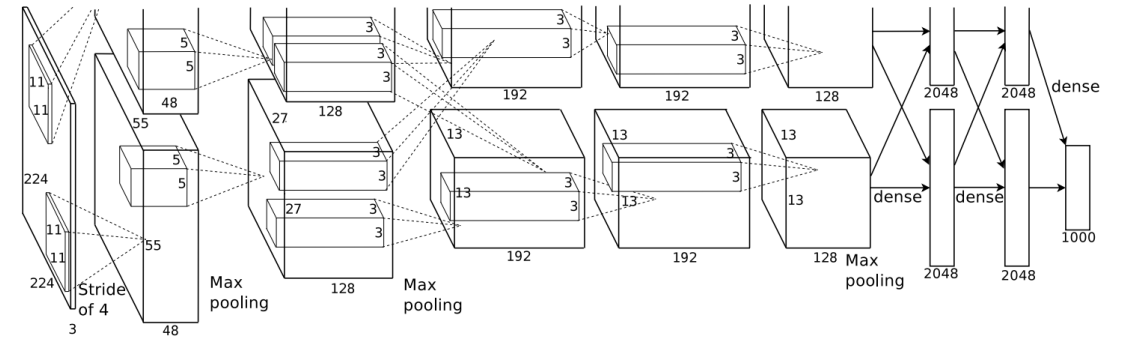
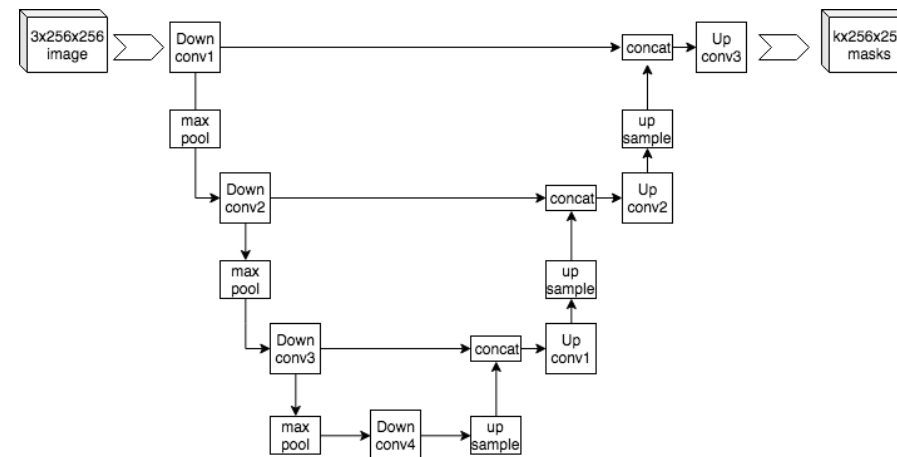
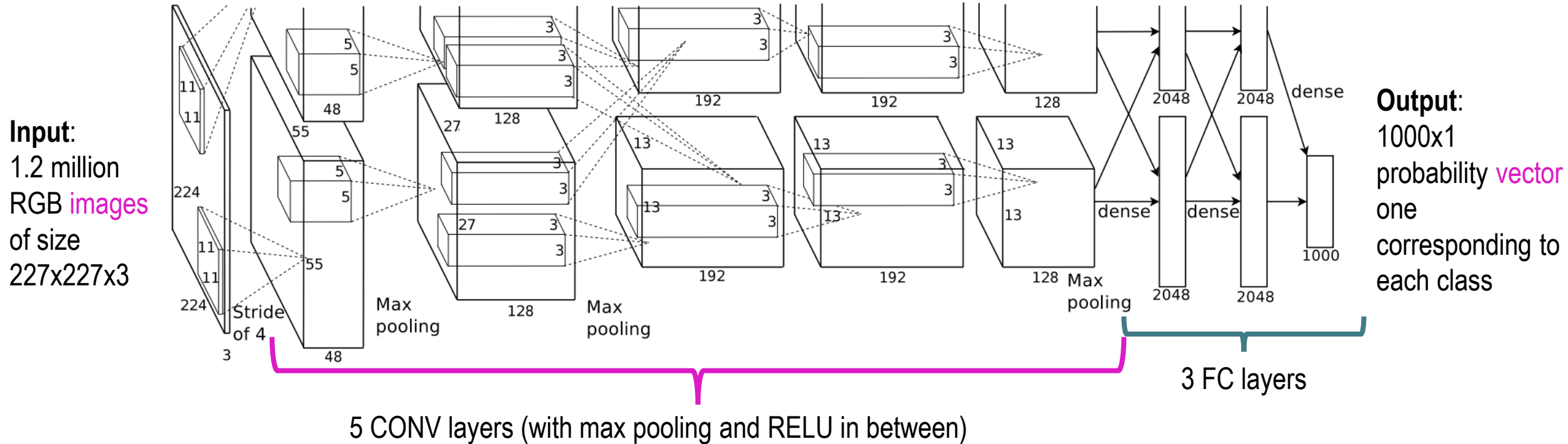


Image to Image



# AlexNet - record-breaking results! “Depth really is important for achieving our results”

[AlexNet by Krizhevsky et al. NeurIPS 2012, ImageNet Classification with Deep Convolutional Neural Networks]



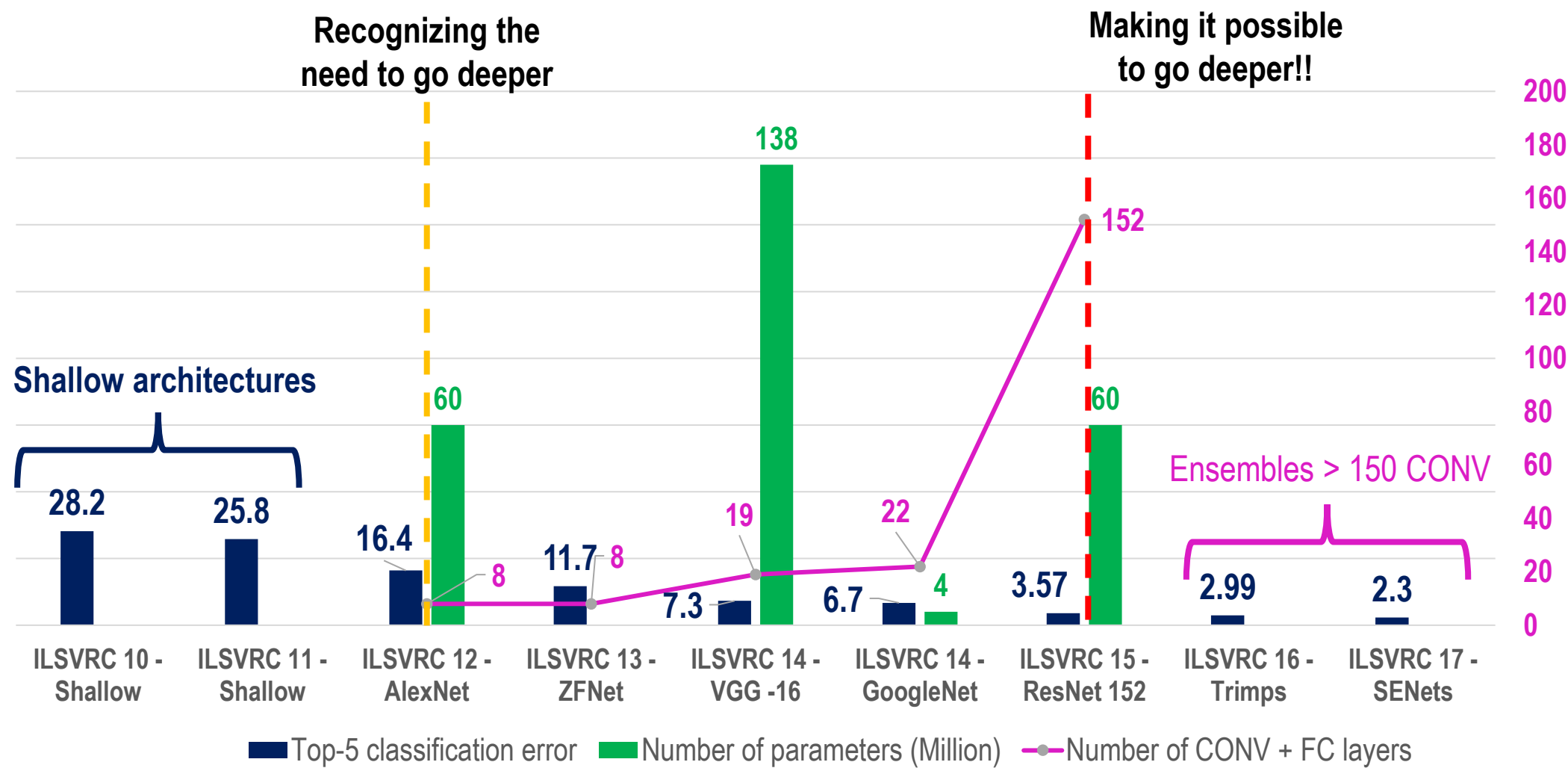
| AlexNet Network - Structural Details |        |     |             |    |      |      |            |
|--------------------------------------|--------|-----|-------------|----|------|------|------------|
| Layer                                | Stride | Pad | Kernel size |    | in   | out  | # of Param |
| conv1                                | 4      | 0   | 11          | 11 | 3    | 96   | 34944      |
| maxpool1                             | 2      | 0   | 3           | 3  | 96   | 96   | 0          |
| conv2                                | 1      | 2   | 5           | 5  | 96   | 256  | 614656     |
| maxpool2                             | 2      | 0   | 3           | 3  | 256  | 256  | 0          |
| conv3                                | 1      | 1   | 3           | 3  | 256  | 384  | 885120     |
| conv4                                | 1      | 1   | 3           | 3  | 384  | 384  | 1327488    |
| conv5                                | 1      | 1   | 3           | 3  | 384  | 256  | 884992     |
| maxpool5                             | 2      | 0   | 3           | 3  | 256  | 256  | 0          |
| fc6                                  |        |     | 1           | 1  | 9216 | 4096 | 37752832   |
| fc7                                  |        |     | 1           | 1  | 4096 | 4096 | 16781312   |
| fc8                                  |        |     | 1           | 1  | 4096 | 1000 | 4097000    |
| Total                                |        |     |             |    |      |      | 62,378,344 |

## Key features:

- Data augmentation (mirroring and cropping) by training on multiple GPUs
- **RELU activation functions** (led to 25% less error rate, and x6 faster compared to when TANH was used)
- use of Local Response Normalization (normalization in the neighborhood of pixels amplifying the excited neuron, while dampening surrounding neurons)

← Around 60M parameters to learn!!!

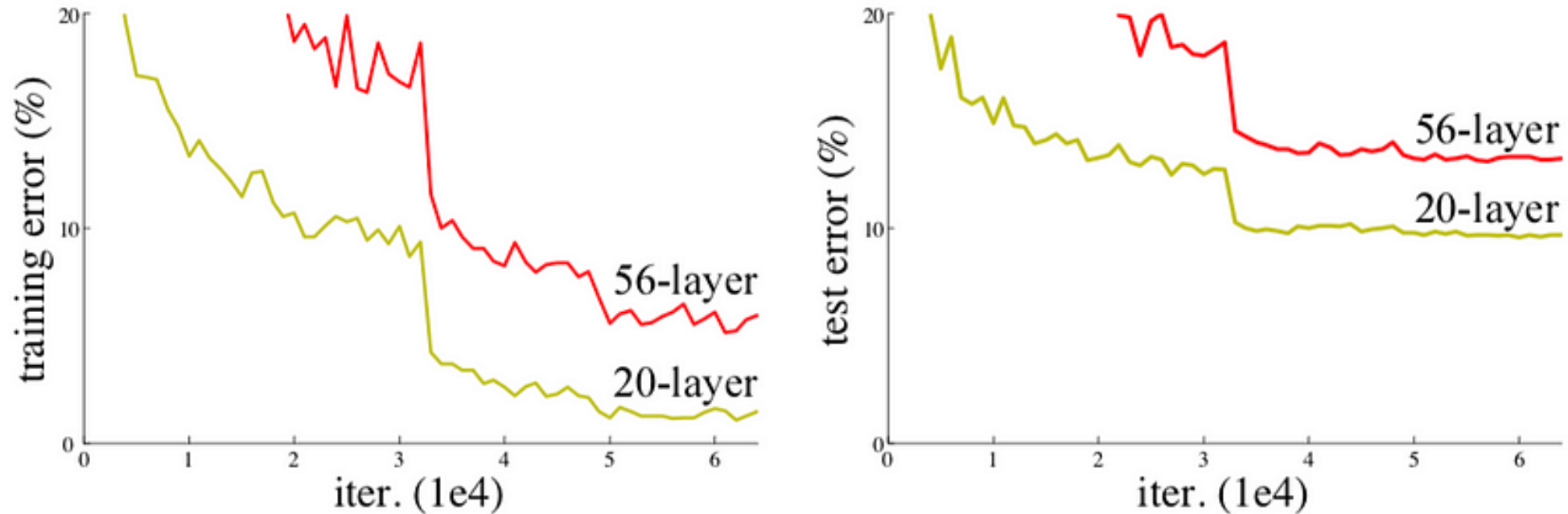
# Image classification challenge at the ILSVRC competition - ImageNet dataset



2017! Top-5 classification error drops to 2.3% and the competition closed! (considered a solved problem)

# Going deeper does not always help: vanishing gradients

From: <https://arxiv.org/pdf/1512.03385.pdf>



As the gradient is backpropagated to earlier layers, repeated multiplication may make the **gradient infinitely small** → the deeper the network, the more its performance becomes saturated or rapidly degrading.

# The Residual Unit idea

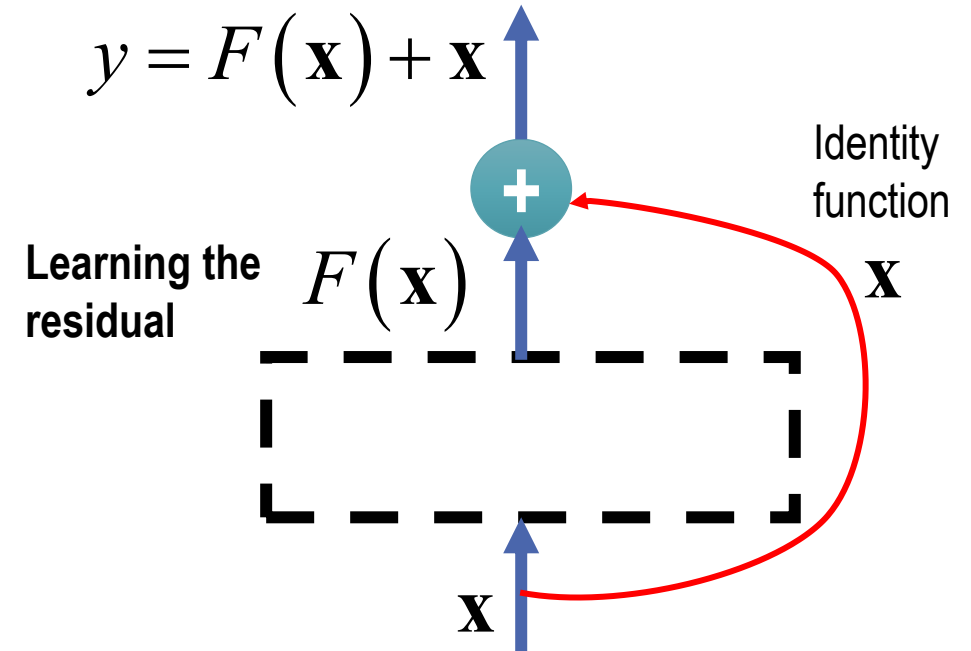
<https://arxiv.org/pdf/1512.03385.pdf>

- Rather than having a Layer that transform  $\mathbf{x}$ , consider a layer **that adds** something - **a residual** - to the previous value, so that

$$y = \mathbf{x} + F(\mathbf{x})$$

**F is the residual function** (i.e. difference between input and output)

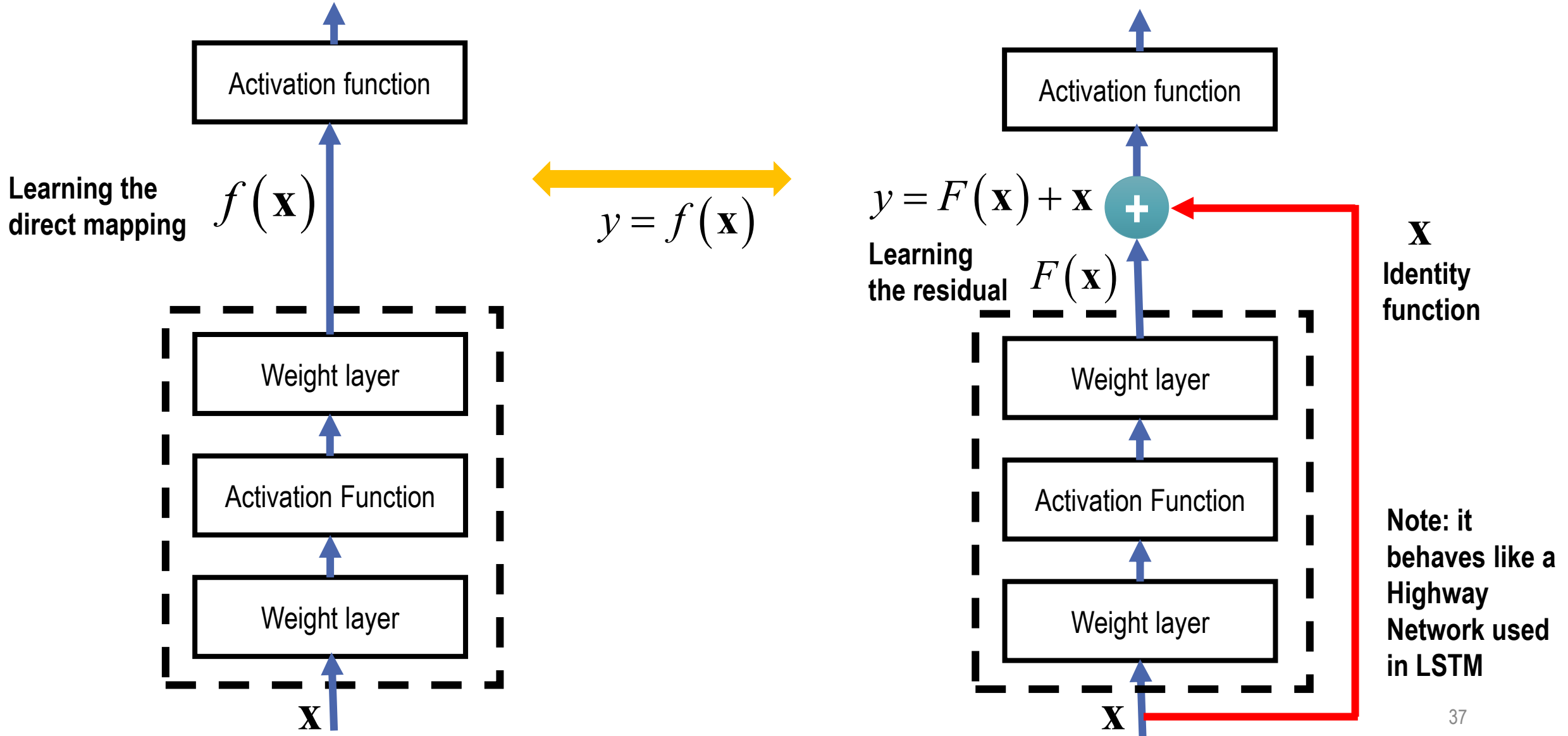
- We **could** learn the parameters of  $F$  so that  $F(\mathbf{x})=\text{zero}$ 
  - $\mathbf{x}$  would not change at all!
  - This residual block would then represent the **identity function**, that is easier to learn since it does not introduce extra parameters to the direct mapping 😊
- **Key difference:** learning the residual mapping rather than the desired underlying mapping!



$F$  is a **shallow neural network** for which we will compute the weights and biases! So, we do not have to worry anymore with vanishing gradients!

# Difference between a “regular block” in CNN and a typical **residual block layer**

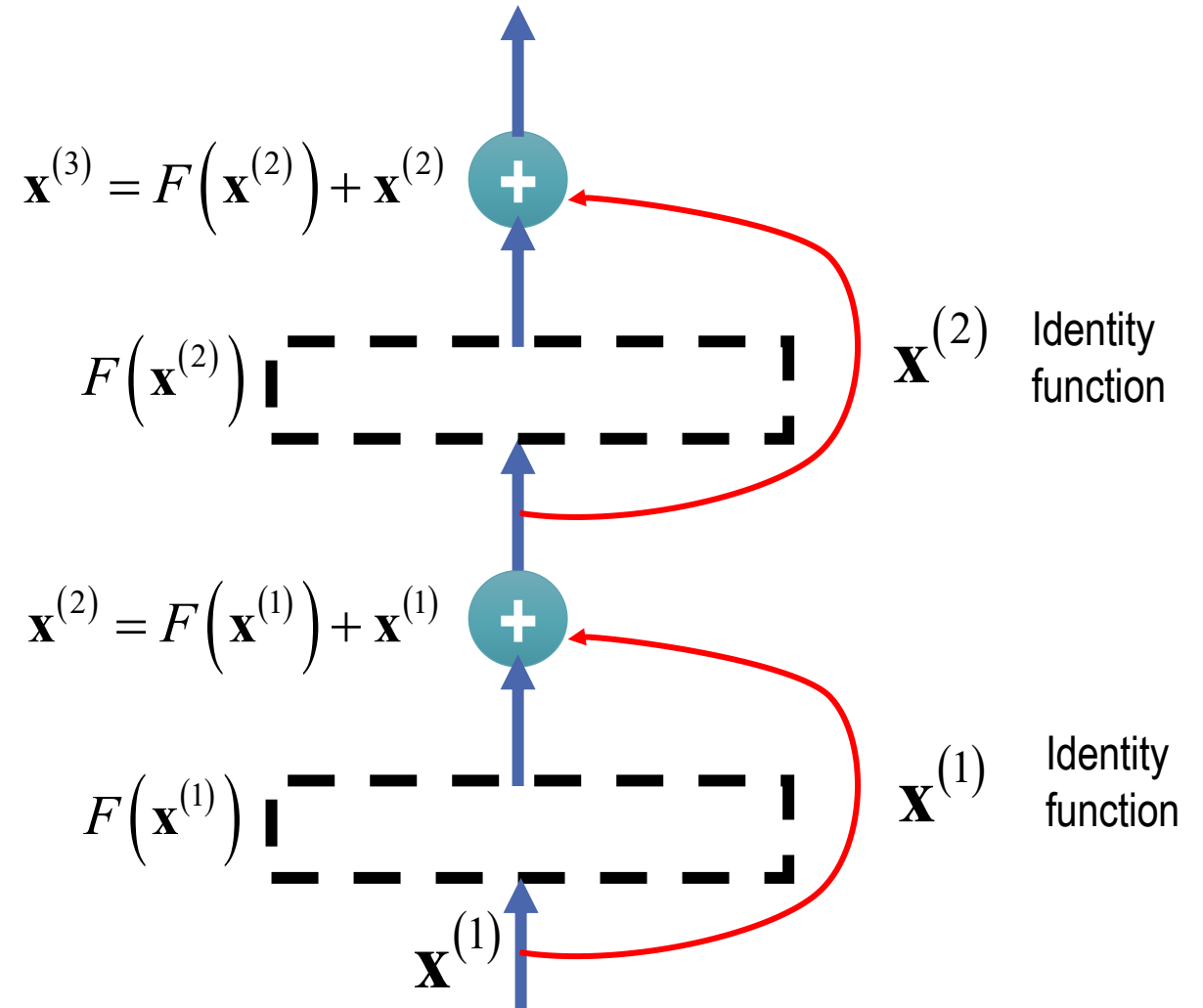
**Weight layer:** fully connected layer or a convolutional layer where we learn the weights and biases





# ResNET (Residual Network): a stack of Residual Block (RB) layers to go deeper! <https://arxiv.org/pdf/1512.03385.pdf>

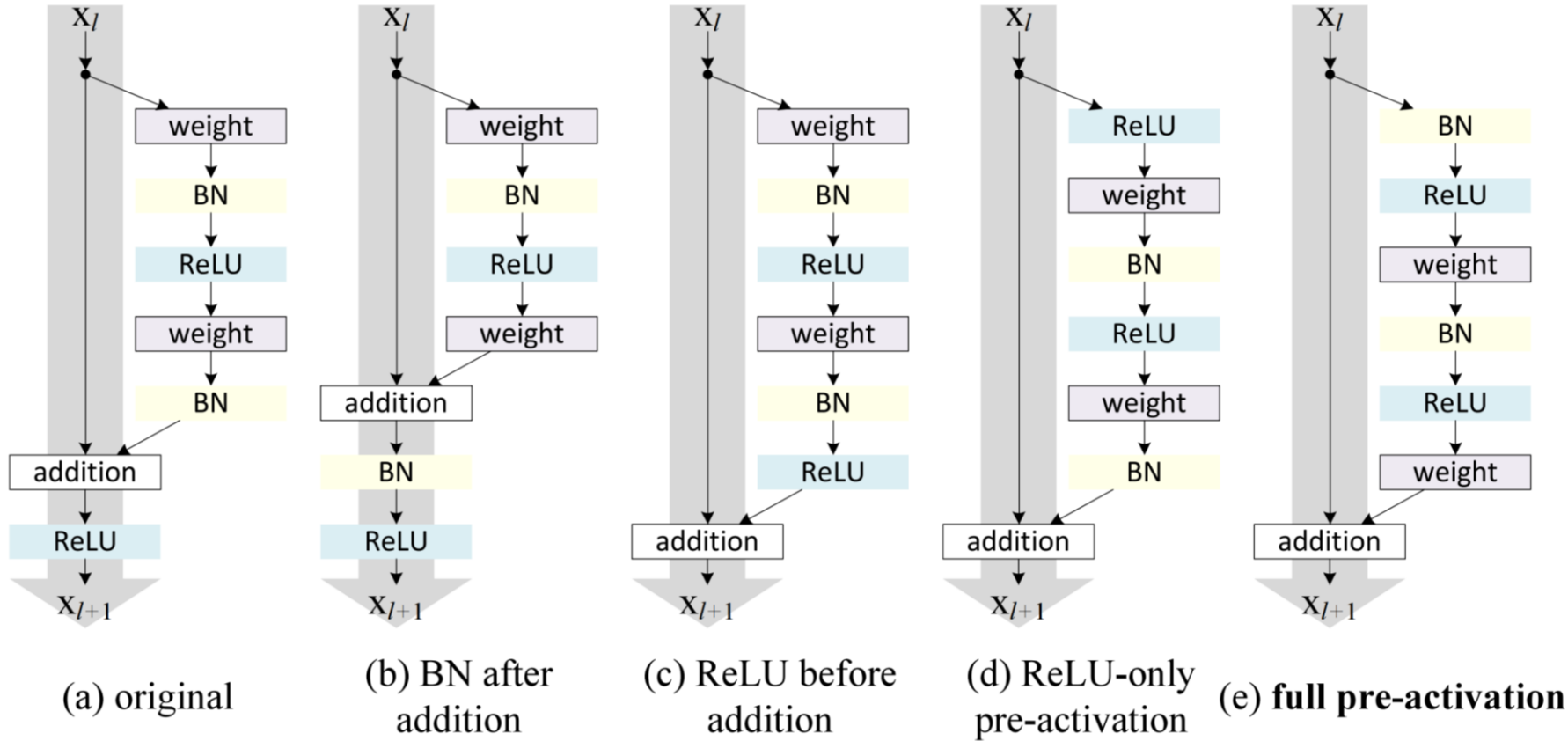
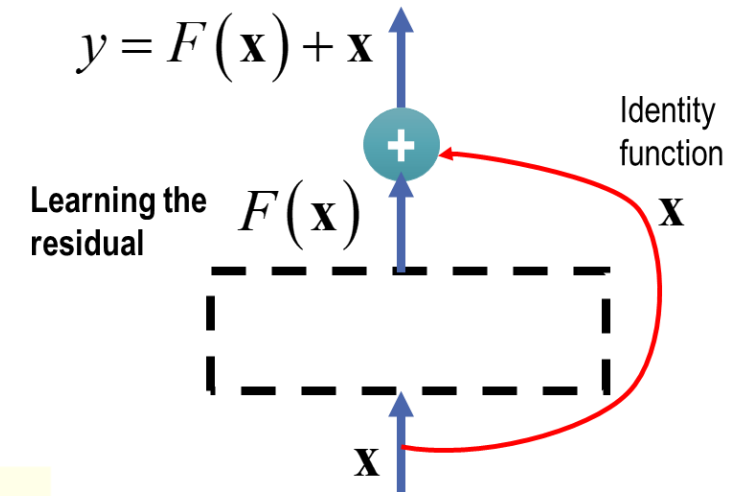
- Each **RB layer** is now computing a separate residual function with **separate trainable parameters** (weights and biases).
- As long as each **RB layer** has a shallow architecture, we can stop worrying about vanishing gradients!
- This smooths out the gradient flow during backpropagation, enabling the network to scale to 50, 100, or even 150 layers.
- **Many state-of-the-art CNN architectures (using data beyond images!) are employing elements of the ResNet for tasks beyond image classification!**



# Alternative ways to build the residual block layer

(aka identity shortcut connection or skip units)

from: <https://arxiv.org/abs/1603.05027>



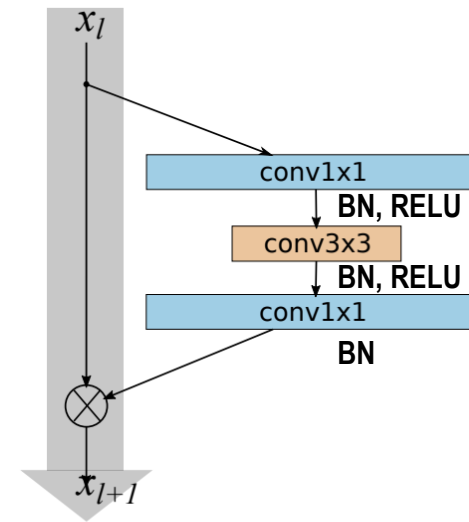
**Weight layer:** fully connected layer or a convolutional layer where we learn the weights and biases

Note: to perform the addition operation,  $F$  and  $x$  must have the same size!

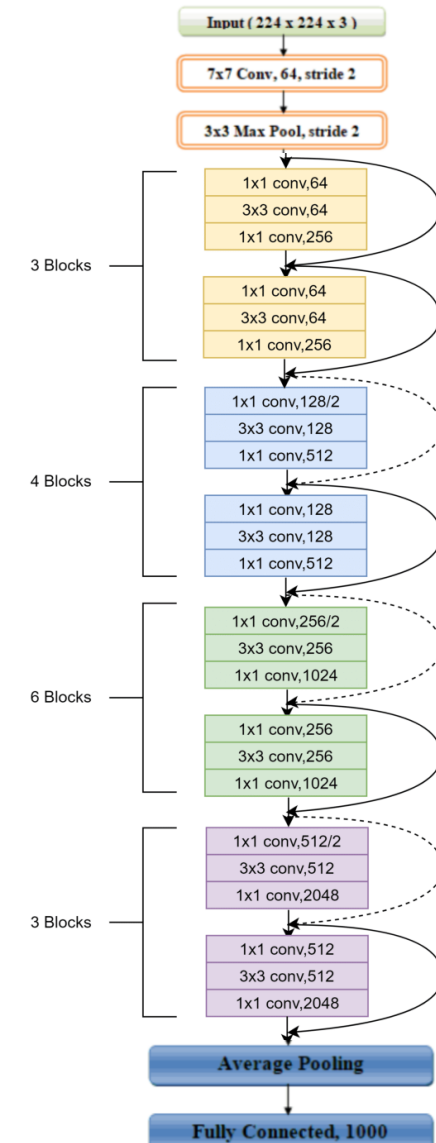
**Problem:** in CNN we start with an image of a certain size, and we change it via convolution (e.g. half the HxW but double the W)!

**Solution:** bottleneck design using

- **Zero-padding (solid line):** extra zero entries should be padded for increasing dimensions. This is not going to introduce any extra parameter.
- **Projected shortcuts (dashed lines):** for each residual function, 3 layers are stacked one over the other:
  - 1 x 1 CONV to reduce dimensions
  - 3 x 3 CONV bottleneck with smaller input/dimensions
  - 1 x 1 CONV to restore dimensions



ResNet-50 layers architecture



# The **up-convolution** (or deconvolution or convolution transpose) **layer**

Restore the condensed feature map to the original size of the input image → expand the feature dimensions.

## Convolution

### Input feature maps

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 0   | 0   | 100 | 100 |
| 0   | 200 | 100 | 100 | 100 |
| 100 | 0   | 100 | 100 | 100 |
| 0   | 0   | 0   | 0   | 100 |
| 0   | 0   | 0   | 0   | 100 |

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 200 | 100 | 150 | 150 |
| 100 | 0   | 0   | 250 | 0   |
| 100 | 0   | 0   | 0   | 250 |
| 0   | 0   | 0   | 0   | 0   |
| 0   | 0   | 0   | 0   | 0   |

|     |     |   |   |     |
|-----|-----|---|---|-----|
| 200 | 100 | 0 | 0 | 0   |
| 100 | 200 | 0 | 0 | 0   |
| 100 | 100 | 0 | 0 | 0   |
| 0   | 0   | 0 | 0 | 100 |
| 0   | 0   | 0 | 0 | 100 |



### Output feature map

|       |  |  |
|-------|--|--|
| OM1,1 |  |  |
|       |  |  |
|       |  |  |

## Up-Convolution

### Input feature map

|       |  |  |
|-------|--|--|
| OM1,1 |  |  |
|       |  |  |
|       |  |  |



### Output feature maps

|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 0   | 0   | 100 | 100 |
| 0   | 200 | 100 | 100 | 100 |
| 100 | 0   | 100 | 100 | 100 |
| 0   | 0   | 0   | 0   | 100 |
| 0   | 0   | 0   | 0   | 100 |

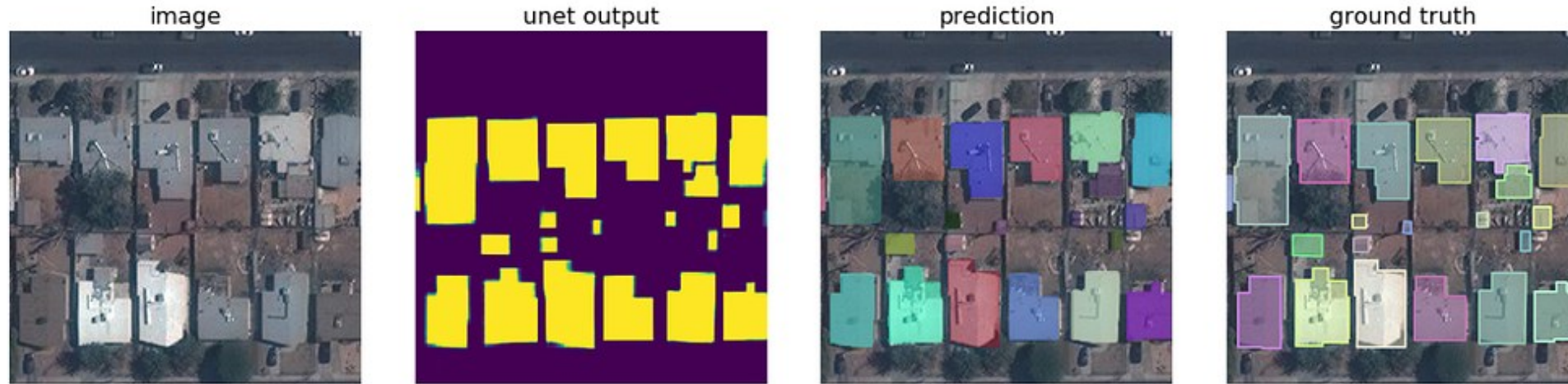
|     |     |     |     |     |
|-----|-----|-----|-----|-----|
| 100 | 200 | 100 | 150 | 150 |
| 100 | 0   | 0   | 250 | 0   |
| 100 | 0   | 0   | 0   | 250 |
| 0   | 0   | 0   | 0   | 0   |
| 0   | 0   | 0   | 0   | 0   |

|     |     |   |   |     |
|-----|-----|---|---|-----|
| 200 | 100 | 0 | 0 | 0   |
| 100 | 200 | 0 | 0 | 0   |
| 100 | 100 | 0 | 0 | 0   |
| 0   | 0   | 0 | 0 | 100 |
| 0   | 0   | 0 | 0 | 100 |

Similarly to convolution, the up-convolution requires learning the weights and biases

# Image segmentation task using the U-Net

<https://arxiv.org/abs/1505.04597>



**Semantic segmentation** output: a high-resolution image, in which each **pixel** is classified.

**Instance segmentation** output: classifying each **instance** of the same class separately

- U-Net is a CNN architecture that is well-established for **localising and distinguishing borders**, by doing classification on every pixel (preserves fine-scale information while learning multi-scale features)
- This means that the **input and output share the same size!**
- Effective even with a limited data set.
- **Many state-of-the-art CNN architectures (using data beyond images!) are employing elements of the U-Net for tasks beyond image segmentation! (e.g. classification task by adding an MLP at the end of u-net architecture to produce class labels)**

Example pictures: <https://github.com/neptune-ai/open-solution-mapping-challenge>

# The contracting and expansive paths determining the U-shape in U-Net:

encoder-decoder architecture with skip connections

- **Contracting:** “**what**” not “where”
  - repeated application of **convolutions (stride 1)**, each followed by a **RELU** and a **max pooling** operation.
  - the spatial resolution is reduced while feature information and number of channels are increased.
- **Expansive:** “**where**”, combined with “**what**”
  - combines the feature and spatial information through a sequence of **up-convolutions (stride 2)** and **concatenations with high-resolution features from the contracting path**
  - **Skip connections** (grey arrows): to combine information from the contractive path to the corresponding level of the expansive path.
  - Spatial resolution is increasing, # of channels decreasing.
  - Yields the segmentation map (same size as input)

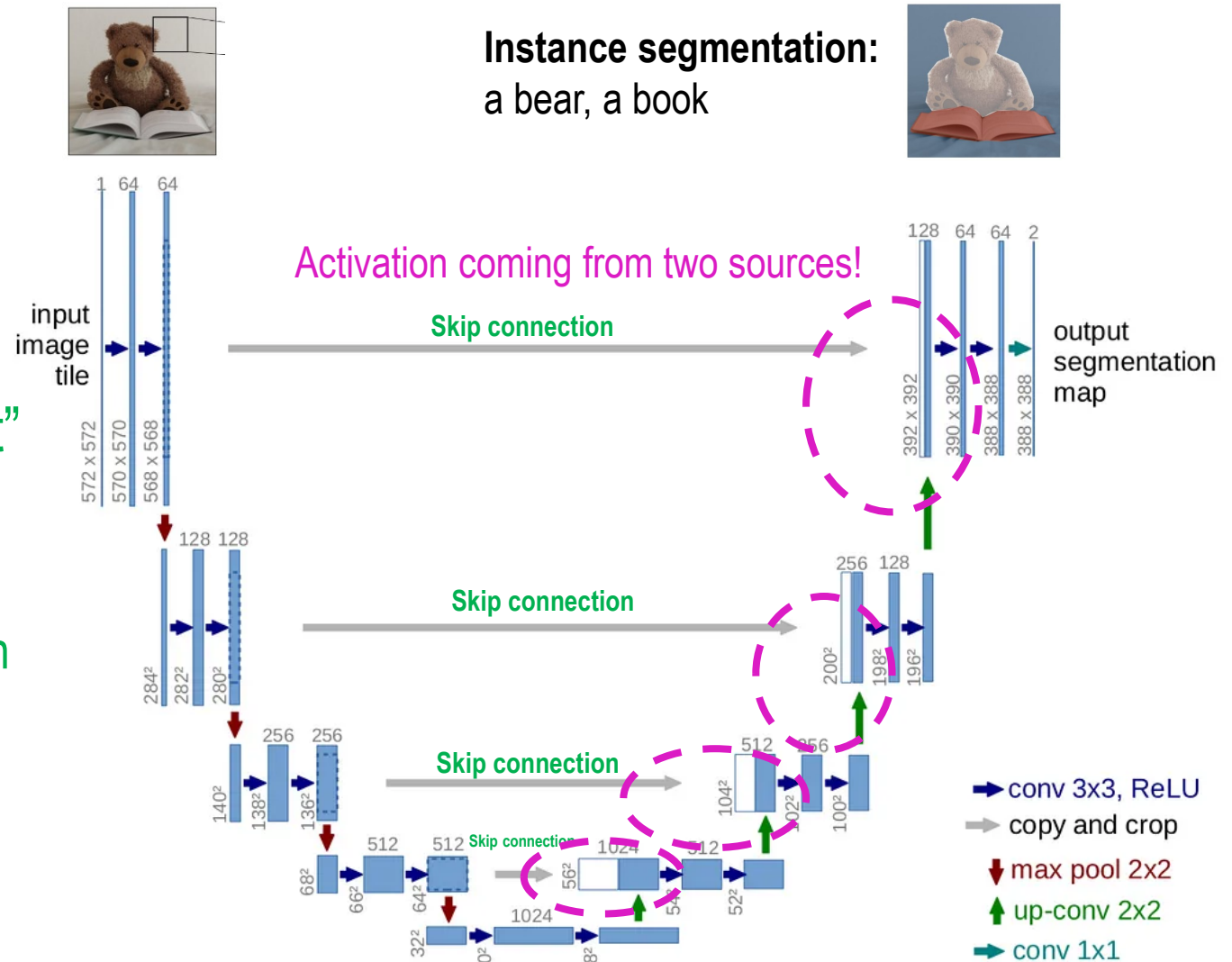


Figure from: <https://arxiv.org/abs/1505.04597>



# CNN are widely applied beyond images: including measurements, time series\*, words

## Data expressed as a tensor (a multidimensional array)

- **2D tensor:** a **vector** in N dimensions. Some examples:
  - **One-dimensional (spatial or temporal) sequence of data** obtained from measurements is reshaped, usually subdivided into **multiple input/output patterns called samples**; (e.g. <https://machinelearningmastery.com/how-to-develop-convolutional-neural-network-models-for-time-series-forecasting/>).
  - **words represented as vectors**;
- **3D tensor:** a **matrix** in N dimension.  
E.g. 2-dimensional data subdivided in multiple channels:  
**Images.**
- **4D tensor:** a **matrix** in MxN dimensions.  
E.g. **video** (sequence of images), 3D medical images



**Build model predictor for a wide-range of tasks:**  
classification, regression,  
detection, segmentation, image  
and video recognition, time-  
series analysis, text-  
classification, ...

Deep Learning algorithms that from an input tensor use learnable weights and biases to automatically learn filters which extract features needed for to build a predictive model.

# Today you learned about

- Convolution Neural Network (**CNN**) architectures
- Residual Neural Network (**ResNet**)
- U-net
- And the fact that **they can be used beyond image analysis..**  
Since it is all about TENSORS !

## The material presented in based on

- **Deep Learning: foundations and concepts, C. Bishop, H. Bishop**  
(<https://www.bishopbook.com/>)
- **Pattern Recognition and machine learning, C. Bishop**  
(<https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>)
- **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, third edition, Aurélien Géron**
- **Excellent online resources with coding examples:**
  - <https://machinelearningmastery.com/>

I attempted to credit all the content/figures where I could find the original source. Sometimes the material is so widely used that it's difficult to be sure of the original author. Please let me know if you find missing references.



# **Bonus slide:** Things you should **never forget** when building your CNN for image analysis!

- **Data Preparation:** Standardize input data - both the dimensions and pixels.
- **Deeper is not always the solution!**
- **Optimizing the architecture is key:**
  - Make sure you decrease the HxWxD slowly!
  - CNN are mostly made of 4 layers types: CONV, POOL, FC, RELU
  - Include BN layers to standardize each feature channel within the batch
  - Consider stacks of small filters (e.g. 3x3) with RELU in-between
- **Memory bottleneck.** Keep track of
  - **Intermediate volume sizes:** determine the raw number of activations per layer and gradients
  - **Parameter and hyperparameters sizes** → this will affect the gradients
  - **Miscellaneous memory**
- **Rather than starting with a CNN architecture from scratch, consider using a pre-trained CNN model**